```
import { useState, useEffect, useRef, useCallback, useMemo } from "react";
import { FileText, BookOpen, Star, Volume2, Sun, Moon, Plus, X, Globe, ChevronDow

// ── TTS helpers ──────────────────────────────────────────────────────────────

function primeVoices() {
  if (typeof window === "undefined" || !("speechSynthesis" in window)) return;
  const synth = window.speechSynthesis;
  synth.getVoices();
  if (typeof synth.onvoiceschanged !== "undefined") {
    synth.onvoiceschanged = () => { synth.getVoices(); };
  }
}
if (typeof window !== "undefined") primeVoices();

function pickVoice(lang: string): SpeechSynthesisVoice | null {
  if (!("speechSynthesis" in window)) return null;
  const voices = window.speechSynthesis.getVoices();
  if (!voices.length) return null;
  const base = lang.split("-")[0].toLowerCase();
  return (
    voices.find(v => v.lang.toLowerCase() === lang.toLowerCase()) ||
    voices.find(v => v.lang.toLowerCase().replace("_", "-").startsWith(base + "-"
    voices.find(v => v.lang.toLowerCase().startsWith(base)) ||
    null
  );
}

function chunkText(text: string, max = 190): string[] {
  const t = text.trim();
  if (t.length <= max) return [t];
  const parts = t.split(/([.!?؟،,;:\n])/);
  const chunks: string[] = [];
  let buf = "";
  for (const p of parts) {
    if ((buf + p).length > max) {
      if (buf.trim()) chunks.push(buf.trim());
      buf = p;
    } else {
      buf += p;
    }
  }
  if (buf.trim()) chunks.push(buf.trim());
  return chunks.length ? chunks : [t.slice(0, max)];
```

```
}

let ttsAudio: HTMLAudioElement | null = null;
function stopTtsAudio() {
  if (ttsAudio) {
    try { ttsAudio.pause(); ttsAudio.src = ""; } catch { /* ignore */ }
    ttsAudio = null;
  }
}

// Global TTS speed: 0.6 = slow, 1.0 = normal, 1.5 = fast
let globalTtsRate: number = 1.0;

function googleTtsUrl(text: string, langShort: string): string {
  return `https://translate.google.com/translate_tts?ie=UTF-8&q=${encodeURICompon
}

function speakViaAudio(text: string, ttsLang: string) {
  const langShort = ttsLang.split("-")[0].toLowerCase();
  const chunks = chunkText(text);
  stopTtsAudio();
  const el = new Audio();
  el.preload = "auto";
  ttsAudio = el;
  let i = 0;
  const playNext = () => {
    if (ttsAudio !== el || i >= chunks.length) { if (ttsAudio === el) ttsAudio =
    el.src = googleTtsUrl(chunks[i], langShort);
    el.play().catch(() => { i++; playNext(); });
  };
  el.onended = () => { i++; playNext(); };
  el.onerror = () => { i++; playNext(); };
  playNext();
}

function speak(text: string, ttsLang: string) {
  if (!text) return;
  const synth = (typeof window !== "undefined" && "speechSynthesis" in window) ?
  if (synth && typeof synth.speak === "function") {
    try {
      if (synth.paused) synth.resume();
      if (synth.speaking || synth.pending) synth.cancel();
      const u = new SpeechSynthesisUtterance(text);
      u.lang = ttsLang;
      u.rate = globalTtsRate * (ttsLang.startsWith("ar") ? 0.9 : 0.95);
      u.pitch = 1;
      u.volume = 1;
```

```
      const v = pickVoice(ttsLang);
      if (v) u.voice = v;
      let started = false;
      u.onstart = () => { started = true; };
      u.onerror = () => { if (!started) speakViaAudio(text, ttsLang); };
      synth.speak(u);
      setTimeout(() => { if (!started) { try { synth.cancel(); } catch { /* ignor
      return;
    } catch { /* ignore */ }
  }
  speakViaAudio(text, ttsLang);
}

function SpeakBtn({ text, ttsLang, size = 14, color = "#60a5fa" }: { text: string
  return (
    <button
      onClick={(e) => { e.stopPropagation(); speak(text, ttsLang); }}
      title="Listen"
      style={{
        display: "inline-flex", alignItems: "center", justifyContent: "center",
        width: size + 14, height: size + 14, borderRadius: "50%",
        border: `1px solid ${color}33`, background: `${color}15`, color,
        cursor: "pointer", padding: 0, flexShrink: 0,
      }}
    >
      <Volume2 size={size} />
    </button>
  );
}

// ─── Theme ──────────────────────────────────────────────────────────────

const DARK = {
  bg: "#0c0f14", card: "#111827", card2: "#0d1117", border: "#1e2533",
  text: "#e2e8f0", textMuted: "#94a3b8", textDim: "#475569", textFaint: "#334155"
  textGhost: "#2d3748", navBg: "#0c0f14", inputBg: "#111827", inputBorder: "#1e2d
  pillBg: "#0c0f14", overlay: "#000000b0", overlayFull: "#0c0f14", sheetBg: "#111
};
const LIGHT = {
  bg: "#f1f5f9", card: "#ffffff", card2: "#f8fafc", border: "#e2e8f0",
  text: "#0f172a", textMuted: "#334155", textDim: "#64748b", textFaint: "#94a3b8"
  textGhost: "#cbd5e1", navBg: "#ffffff", inputBg: "#ffffff", inputBorder: "#cbd5
  pillBg: "#f8fafc", overlay: "#00000060", overlayFull: "#f1f5f9", sheetBg: "#fff
};
type Theme = typeof DARK;

// ─── Language registry ──────────────────────────────────────────────────
```

```
type LanguageInfo = {
  code: string;          // Google translate code
  name: string;          // English name
  nativeName: string;    // Name in own language
  flag: string;          // Emoji flag
  dir: "ltr" | "rtl";
  ttsCode: string;       // BCP-47 for TTS
};

const LANGUAGES: Record<string, LanguageInfo> = {
  en:      { code: "en",      name: "English",              nativeName: "English"
  ar:      { code: "ar",      name: "Arabic",               nativeName: "العربية"
  fi:      { code: "fi",      name: "Finnish",              nativeName: "Suomi",
  es:      { code: "es",      name: "Spanish",              nativeName: "Español"
  fr:      { code: "fr",      name: "French",               nativeName: "Français
  de:      { code: "de",      name: "German",               nativeName: "Deutsch"
  it:      { code: "it",      name: "Italian",              nativeName: "Italiano
  pt:      { code: "pt",      name: "Portuguese",           nativeName: "Português
  nl:      { code: "nl",      name: "Dutch",                nativeName: "Nederlan
  sv:      { code: "sv",      name: "Swedish",              nativeName: "Svenska"
  no:      { code: "no",      name: "Norwegian",            nativeName: "Norsk",
  da:      { code: "da",      name: "Danish",               nativeName: "Dansk",
  ru:      { code: "ru",      name: "Russian",              nativeName: "Русский"
  pl:      { code: "pl",      name: "Polish",               nativeName: "Polski",
  tr:      { code: "tr",      name: "Turkish",              nativeName: "Türkçe",
  ja:      { code: "ja",      name: "Japanese",             nativeName: "日本語",
  ko:      { code: "ko",      name: "Korean",               nativeName: "한국어",
  "zh-CN": { code: "zh-CN",   name: "Chinese (Simplified)", nativeName: "中文（简体
  "zh-TW": { code: "zh-TW",   name: "Chinese (Traditional)",nativeName: "中文（繁體
  hi:      { code: "hi",      name: "Hindi",                nativeName: "हिन्दी",
  he:      { code: "iw",      name: "Hebrew",               nativeName: "עברית",
  fa:      { code: "fa",      name: "Persian",              nativeName: "فارسی",
  ur:      { code: "ur",      name: "Urdu",                 nativeName: "اردو",
  el:      { code: "el",      name: "Greek",                nativeName: "Ελληνικά
  cs:      { code: "cs",      name: "Czech",                nativeName: "Čeština"
  hu:      { code: "hu",      name: "Hungarian",            nativeName: "Magyar",
  ro:      { code: "ro",      name: "Romanian",             nativeName: "Română",
  uk:      { code: "uk",      name: "Ukrainian",            nativeName: "Українсь
  vi:      { code: "vi",      name: "Vietnamese",           nativeName: "Tiếng Vi
  th:      { code: "th",      name: "Thai",                 nativeName: "ไทย",
  id:      { code: "id",      name: "Indonesian",           nativeName: "Bahasa I
  ms:      { code: "ms",      name: "Malay",                nativeName: "Bahasa M
  bg:      { code: "bg",      name: "Bulgarian",            nativeName: "Българск
  hr:      { code: "hr",      name: "Croatian",             nativeName: "Hrvatski
  sk:      { code: "sk",      name: "Slovak",               nativeName: "Slovenči
  sl:      { code: "sl",      name: "Slovenian",            nativeName: "Slovenšč
```

```
  et:      { code: "et",      name: "Estonian",             nativeName: "Eesti",
  lv:      { code: "lv",      name: "Latvian",              nativeName: "Latviešu
  lt:      { code: "lt",      name: "Lithuanian",           nativeName: "Lietuvių
};

const LANG_CODES = Object.keys(LANGUAGES);

function getLang(code: string): LanguageInfo {
  return LANGUAGES[code] || { code, name: code.toUpperCase(), nativeName: code.to
}

// ─── i18n: UI strings ──────────────────────────────────────────────────

type UiLang = "en" | "ar" | "fi" | "es" | "fr";
const UI_LANGS: UiLang[] = ["en", "ar", "fi", "es", "fr"];

const STR: Record<UiLang, Record<string, string>> = {
  en: {
    pasteHint: "Paste text in any language — translation is automatic.",
    pastePlaceholder: "Paste your text here...",
    sourceLang: "Source language",
    autoDetect: "Auto-detect",
    translateTo: "Translate to",
    addLanguage: "Add language",
    chooseLanguage: "Choose a language…",
    readNow: "Read now",
    newText: "New text",
    colorize: "Colorize words by type",
    colorizeOn: "Colors on",
    colorizeOff: "Colors off",
    quiz: "Quiz",
    quizSetupTitle: "How many words to study today?",
    quizMode: "Quiz mode",
    modeTyping: "Type the answer",
    modeChoice: "Multiple choice",
    modeReverse: "Reverse",
    quizStart: "Start quiz",
    quizCheck: "Check",
    quizNext: "Next →",
    quizSkip: "Skip",
    quizHint: "Hint",
    quizYourAnswer: "Your answer",
    quizCorrect: "Correct answer",
    quizMarkRight: "Mark as correct",
    quizFinishTitle: "Great job!",
    quizScore: "Score",
    quizAccuracy: "Accuracy",
```

```
quizCorrectCount: "Correct",
quizWrongCount: "Wrong",
quizReviewWrong: "Review wrong words",
quizDone: "Done",
quizAll: "All",
quizIgnoreDiacritics: "Ignore Arabic diacritics",
needsReview: "Needs review",
edited: "Edited",
addNote: "Add note",
yourNote: "Your note",
saveNote: "Save",
cancel: "Cancel",
editTranslation: "Edit translation",
saveEdit: "Save",
resetOriginal: "Reset to original",
exampleSentence: "Example",
translateSentence: "Translate sentence",
searchSaved: "Search saved words…",
filterAll: "All",
filterReview: "🔁 Review",
filterEdited: "✓ Edited",
sortNewest: "Newest",
sortAlpha: "A→Z",
sortMostWrong: "Most wrong",
streak: "day streak",
dailyGoal: "Today's goal",
soundOn: "Sound on",
soundOff: "Sound off",
timer: "Timer",
quizCelebrateGreat: "Excellent! 🌟",
quizCelebrateGood: "Well done! 👏",
quizCelebrateKeep: "Keep going! 💪",
text: "Text",
reader: "Reader",
saved: "Saved",
translate: "Translate",
translating: "Translating…",
typeOrTap: "Click 'Translate' or type here…",
tapAnyWord: "Tap any word to translate it",
pronunciation: "Pronunciation",
synonym: "Synonym",
type: "Type",
save: "Save word",
remove: "Remove",
flashcard: "Flashcard",
download: "Download",
deleteAll: "Delete all",
```

```
  nothingSaved: "No saved words yet. Tap a word and choose Save.",
  countSaved: "saved words",
  next: "Next",
  prev: "Previous",
  learned: "Learned",
  close: "Close",
  tapToTranslate: "Tap a language to see translation",
  noTranslation: "No saved translation",
  copyAll: "Copy all",
  copied: "Copied!",
  savedWordsTitle: "Saved Words",
  light: "Light mode",
  dark: "Dark mode",
  uiLanguage: "Interface language",
  noFlashcards: "No saved words to study.",
},
ar: {
  pasteHint: "الصق نصاً بأي لغة — الترجمة تلقائية.",
  pastePlaceholder: "الصق نصك هنا...",
  sourceLang: "لغة المصدر",
  autoDetect: "كشف تلقائي",
  translateTo: "ترجم إلى",
  addLanguage: "أضف لغة",
  chooseLanguage: "اختر لغة...",
  readNow: "اقرأ الآن",
  newText: "نص جديد",
  colorize: "تلوين الكلمات حسب نوعها",
  colorizeOn: "الألوان مفعلة",
  colorizeOff: "الألوان مغلقة",
  quiz: "اختبار",
  quizSetupTitle: "كم كلمة تريد مذاكرتها اليوم؟",
  quizMode: "نمط الاختبار",
  modeTyping: "اكتب الإجابة",
  modeChoice: "اختيار من متعدد",
  modeReverse: "عكسي",
  quizStart: "ابدأ الاختبار",
  quizCheck: "تحقق",
  quizNext: "التالي →",
  quizSkip: "تخطّى",
  quizHint: "تلميح",
  quizYourAnswer: "إجابتك",
  quizCorrect: "الإجابة الصحيحة",
  quizMarkRight: "اعتبرها صحيحة",
  quizFinishTitle: "أحسنت!",
  quizScore: "النتيجة",
  quizAccuracy: "الدقّة",
  quizCorrectCount: "صحيحة",
```

```
quizWrongCount: "خاطئة",
quizReviewWrong: "راجع الكلمات الخاطئة",
quizDone: "تم",
quizAll: "الكل",
quizIgnoreDiacritics: "تجاهل التشكيل",
needsReview: "تحتاج مراجعة",
edited: "مُعدَّلة",
addNote: "أضف ملاحظة",
yourNote: "ملاحظتك",
saveNote: "حفظ",
cancel: "إلغاء",
editTranslation: "عدّل الترجمة",
saveEdit: "حفظ",
resetOriginal: "استرجاع الأصل",
exampleSentence: "مثال",
translateSentence: "ترجم الجملة",
searchSaved: "ابحث في المحفوظات…",
filterAll: "الكل",
filterReview: "🔁 للمراجعة",
filterEdited: "✓ معدّلة",
sortNewest: "الأحدث",
sortAlpha: "أ→ي",
sortMostWrong: "الأكثر خطأً",
streak: "يوم متتالي",
dailyGoal: "هدف اليوم",
soundOn: "الصوت مفعّل",
soundOff: "الصوت مغلق",
timer: "مؤقّت",
quizCelebrateGreat: "🌟 ممتاز!",
quizCelebrateGood: "👏 أحسنت!",
quizCelebrateKeep: "💪 استمر!",
text: "النص",
reader: "القارئ",
saved: "محفوظ",
translate: "ترجم",
translating: "جارٍ الترجمة…",
typeOrTap: "اضغط «ترجم» أو اكتب هنا…",
tapAnyWord: "اضغط على أي كلمة لترجمتها",
pronunciation: "النطق",
synonym: "مرادف",
type: "النوع",
save: "حفظ الكلمة",
remove: "إزالة",
flashcard: "فلاش كارد",
download: "تنزيل",
deleteAll: "حذف الكل",
nothingSaved: "لا توجد كلمات محفوظة بعد. اضغط على كلمة واختر حفظ.",
```

```
  countSaved: "كلمة محفوظة",
  next: "التالي",
  prev: "السابق",
  learned: "تم التعلم",
  close: "إغلاق",
  tapToTranslate: "اضغط على لغة لرؤية الترجمة",
  noTranslation: "لا ترجمة محفوظة",
  copyAll: "نسخ الكل",
  copied: "تم النسخ!",
  savedWordsTitle: "الكلمات المحفوظة",
  light: "الوضع الفاتح",
  dark: "الوضع الداكن",
  uiLanguage: "لغة الواجهة",
  noFlashcards: "لا توجد كلمات محفوظة للدراسة.",
},
fi: {
  pasteHint: "Liitä tekstiä millä tahansa kielellä — käännös on automaattinen."
  pastePlaceholder: "Liitä tekstisi tähän...",
  sourceLang: "Lähdekieli",
  autoDetect: "Tunnista automaattisesti",
  translateTo: "Käännä kielelle",
  addLanguage: "Lisää kieli",
  chooseLanguage: "Valitse kieli…",
  readNow: "Lue nyt",
  newText: "Uusi teksti",
  colorize: "Värjää sanat tyypin mukaan",
  colorizeOn: "Värit päällä",
  colorizeOff: "Värit pois",
  quiz: "Tietokilpailu",
  quizSetupTitle: "Kuinka monta sanaa opiskelet tänään?",
  quizMode: "Tila",
  modeTyping: "Kirjoita vastaus",
  modeChoice: "Monivalinta",
  modeReverse: "Käänteinen",
  quizStart: "Aloita",
  quizCheck: "Tarkista",
  quizNext: "Seuraava →",
  quizSkip: "Ohita",
  quizHint: "Vihje",
  quizYourAnswer: "Vastauksesi",
  quizCorrect: "Oikea vastaus",
  quizMarkRight: "Merkitse oikeaksi",
  quizFinishTitle: "Hienoa työtä!",
  quizScore: "Pisteet",
  quizAccuracy: "Tarkkuus",
  quizCorrectCount: "Oikein",
  quizWrongCount: "Väärin",
```

```
quizReviewWrong: "Kertaa väärät",
quizDone: "Valmis",
quizAll: "Kaikki",
quizIgnoreDiacritics: "Ohita arabian diakriitit",
needsReview: "Vaatii kertausta",
edited: "Muokattu",
addNote: "Lisää muistiinpano",
yourNote: "Muistiinpanosi",
saveNote: "Tallenna",
cancel: "Peruuta",
editTranslation: "Muokkaa käännöstä",
saveEdit: "Tallenna",
resetOriginal: "Palauta alkuperäinen",
exampleSentence: "Esimerkki",
translateSentence: "Käännä lause",
searchSaved: "Hae tallennetuista…",
filterAll: "Kaikki",
filterReview: "🔁 Kerrattavat",
filterEdited: "✓ Muokatut",
sortNewest: "Uusin",
sortAlpha: "A→Ö",
sortMostWrong: "Eniten virheitä",
streak: "päivän putki",
dailyGoal: "Päivän tavoite",
soundOn: "Ääni päällä",
soundOff: "Ääni pois",
timer: "Ajastin",
quizCelebrateGreat: "Erinomaista! 🌟",
quizCelebrateGood: "Hyvin tehty! 👏",
quizCelebrateKeep: "Jatka! 💪",
text: "Teksti",
reader: "Lukija",
saved: "Tallennetut",
translate: "Käännä",
translating: "Käännetään…",
typeOrTap: "Napsauta 'Käännä' tai kirjoita…",
tapAnyWord: "Napsauta sanaa kääntääksesi sen",
pronunciation: "Ääntäminen",
synonym: "Synonyymi",
type: "Tyyppi",
save: "Tallenna sana",
remove: "Poista",
flashcard: "Muistikortti",
download: "Lataa",
deleteAll: "Poista kaikki",
nothingSaved: "Ei tallennettuja sanoja. Napsauta sanaa ja valitse tallenna.",
countSaved: "tallennettua sanaa",
```

```
    next: "Seuraava",
    prev: "Edellinen",
    learned: "Opittu",
    close: "Sulje",
    tapToTranslate: "Napsauta kieltä nähdäksesi käännöksen",
    noTranslation: "Ei tallennettua käännöstä",
    copyAll: "Kopioi kaikki",
    copied: "Kopioitu!",
    savedWordsTitle: "Tallennetut sanat",
    light: "Vaalea tila",
    dark: "Tumma tila",
    uiLanguage: "Käyttöliittymän kieli",
    noFlashcards: "Ei tallennettuja sanoja opiskeltavaksi.",
  },
  es: {
    pasteHint: "Pega texto en cualquier idioma — la traducción es automática.",
    pastePlaceholder: "Pega tu texto aquí...",
    sourceLang: "Idioma de origen",
    autoDetect: "Detección automática",
    translateTo: "Traducir a",
    addLanguage: "Añadir idioma",
    chooseLanguage: "Elige un idioma…",
    readNow: "Leer ahora",
    newText: "Nuevo texto",
    colorize: "Colorear palabras por tipo",
    colorizeOn: "Colores activados",
    colorizeOff: "Colores desactivados",
    quiz: "Examen",
    quizSetupTitle: "¿Cuántas palabras quieres estudiar hoy?",
    quizMode: "Modo",
    modeTyping: "Escribe la respuesta",
    modeChoice: "Opción múltiple",
    modeReverse: "Inverso",
    quizStart: "Empezar",
    quizCheck: "Comprobar",
    quizNext: "Siguiente →",
    quizSkip: "Saltar",
    quizHint: "Pista",
    quizYourAnswer: "Tu respuesta",
    quizCorrect: "Respuesta correcta",
    quizMarkRight: "Marcar como correcta",
    quizFinishTitle: "¡Excelente!",
    quizScore: "Puntuación",
    quizAccuracy: "Precisión",
    quizCorrectCount: "Correctas",
    quizWrongCount: "Erróneas",
    quizReviewWrong: "Repasar erróneas",
```

```
quizDone: "Hecho",
quizAll: "Todas",
quizIgnoreDiacritics: "Ignorar diacríticos árabes",
needsReview: "Necesita repaso",
edited: "Editada",
addNote: "Añadir nota",
yourNote: "Tu nota",
saveNote: "Guardar",
cancel: "Cancelar",
editTranslation: "Editar traducción",
saveEdit: "Guardar",
resetOriginal: "Restaurar original",
exampleSentence: "Ejemplo",
translateSentence: "Traducir frase",
searchSaved: "Buscar guardadas…",
filterAll: "Todas",
filterReview: "🔁 Repaso",
filterEdited: "✓ Editadas",
sortNewest: "Más recientes",
sortAlpha: "A→Z",
sortMostWrong: "Más errores",
streak: "días seguidos",
dailyGoal: "Meta de hoy",
soundOn: "Sonido on",
soundOff: "Sonido off",
timer: "Cronómetro",
quizCelebrateGreat: "¡Excelente! 🌟",
quizCelebrateGood: "¡Bien hecho! 👏",
quizCelebrateKeep: "¡Sigue así! 💪",
text: "Texto",
reader: "Lector",
saved: "Guardado",
translate: "Traducir",
translating: "Traduciendo…",
typeOrTap: "Haz clic en 'Traducir' o escribe…",
tapAnyWord: "Toca cualquier palabra para traducirla",
pronunciation: "Pronunciación",
synonym: "Sinónimo",
type: "Tipo",
save: "Guardar palabra",
remove: "Quitar",
flashcard: "Tarjeta",
download: "Descargar",
deleteAll: "Borrar todo",
nothingSaved: "Aún no hay palabras guardadas. Toca una palabra y elige Guarda
countSaved: "palabras guardadas",
next: "Siguiente",
```

```
    prev: "Anterior",
    learned: "Aprendido",
    close: "Cerrar",
    tapToTranslate: "Toca un idioma para ver la traducción",
    noTranslation: "Sin traducción guardada",
    copyAll: "Copiar todo",
    copied: "¡Copiado!",
    savedWordsTitle: "Palabras Guardadas",
    light: "Modo claro",
    dark: "Modo oscuro",
    uiLanguage: "Idioma de la interfaz",
    noFlashcards: "No hay palabras guardadas para estudiar.",
  },
  fr: {
    pasteHint: "Collez du texte dans n'importe quelle langue — la traduction est
    pastePlaceholder: "Collez votre texte ici...",
    sourceLang: "Langue source",
    autoDetect: "Détection automatique",
    translateTo: "Traduire en",
    addLanguage: "Ajouter une langue",
    chooseLanguage: "Choisir une langue…",
    readNow: "Lire maintenant",
    newText: "Nouveau texte",
    colorize: "Colorer les mots par type",
    colorizeOn: "Couleurs activées",
    colorizeOff: "Couleurs désactivées",
    quiz: "Quiz",
    quizSetupTitle: "Combien de mots étudier aujourd'hui ?",
    quizMode: "Mode",
    modeTyping: "Tapez la réponse",
    modeChoice: "Choix multiple",
    modeReverse: "Inversé",
    quizStart: "Commencer",
    quizCheck: "Vérifier",
    quizNext: "Suivant →",
    quizSkip: "Passer",
    quizHint: "Indice",
    quizYourAnswer: "Votre réponse",
    quizCorrect: "Réponse correcte",
    quizMarkRight: "Marquer correcte",
    quizFinishTitle: "Bravo !",
    quizScore: "Score",
    quizAccuracy: "Précision",
    quizCorrectCount: "Justes",
    quizWrongCount: "Faux",
    quizReviewWrong: "Revoir les erreurs",
    quizDone: "Terminé",
```

```
quizAll: "Tous",
quizIgnoreDiacritics: "Ignorer les diacritiques arabes",
needsReview: "À revoir",
edited: "Modifiée",
addNote: "Ajouter une note",
yourNote: "Votre note",
saveNote: "Enregistrer",
cancel: "Annuler",
editTranslation: "Modifier la traduction",
saveEdit: "Enregistrer",
resetOriginal: "Restaurer l'original",
exampleSentence: "Exemple",
translateSentence: "Traduire la phrase",
searchSaved: "Rechercher…",
filterAll: "Tout",
filterReview: "🔁 À revoir",
filterEdited: "✓ Modifiées",
sortNewest: "Plus récentes",
sortAlpha: "A→Z",
sortMostWrong: "Plus d'erreurs",
streak: "jours d'affilée",
dailyGoal: "Objectif du jour",
soundOn: "Son activé",
soundOff: "Son coupé",
timer: "Minuteur",
quizCelebrateGreat: "Excellent ! 🌟",
quizCelebrateGood: "Bien joué ! 👏",
quizCelebrateKeep: "Continue ! 💪",
text: "Texte",
reader: "Lecteur",
saved: "Enregistré",
translate: "Traduire",
translating: "Traduction…",
typeOrTap: "Cliquez sur 'Traduire' ou tapez ici…",
tapAnyWord: "Touchez un mot pour le traduire",
pronunciation: "Prononciation",
synonym: "Synonyme",
type: "Type",
save: "Enregistrer",
remove: "Retirer",
flashcard: "Carte",
download: "Télécharger",
deleteAll: "Tout supprimer",
nothingSaved: "Aucun mot enregistré. Touchez un mot et choisissez Enregistrer
countSaved: "mots enregistrés",
next: "Suivant",
prev: "Précédent",
```

```
    learned: "Appris",
    close: "Fermer",
    tapToTranslate: "Touchez une langue pour voir la traduction",
    noTranslation: "Aucune traduction enregistrée",
    copyAll: "Tout copier",
    copied: "Copié !",
    savedWordsTitle: "Mots Enregistrés",
    light: "Mode clair",
    dark: "Mode sombre",
    uiLanguage: "Langue de l'interface",
    noFlashcards: "Aucun mot enregistré à étudier.",
  },
};

// ─── POS color map ──────────────────────────────────────────────────────────

const POS_COLORS: Record<string, string> = {
  noun: "#60a5fa",
  verb: "#4ade80",
  adjective: "#f59e0b",
  adverb: "#a78bfa",
  pronoun: "#f472b6",
  preposition: "#34d399",
  conjunction: "#fb7185",
  article: "#94a3b8",
  determiner: "#94a3b8",
  interjection: "#fbbf24",
  numeral: "#22d3ee",
  particle: "#cbd5e1",
  abbreviation: "#cbd5e1",
};

function posColor(pos: string | null | undefined): string {
  if (!pos) return "#60a5fa";
  return POS_COLORS[pos.toLowerCase()] || "#60a5fa";
}

// ─── Emoji dictionary (English keys) ────────────────────────────────────────

const EMOJI_MAP: Record<string, string> = {
  // Animals
  cat: "🐈", kitten: "🐱", dog: "🐕", puppy: "🐶", bird: "🐦", fish: "🐟", horse
  cow: "🐄", pig: "🐖", lion: "🦁", tiger: "🐅", elephant: "🐘", monkey: "🐒", b
  rabbit: "🐇", rat: "🐀", sheep: "🐑", goat: "🐐", chicken: "🐔", duck: "🦆",
  turkey: "🦃", penguin: "🐧", owl: "🦉", eagle: "🦅", parrot: "🦜", peacock: "
  butterfly: "🦋", bee: "🐝", spider: "🕷", snake: "🐍", crocodile: "🐊", turtle
  frog: "🐸", whale: "🐋", dolphin: "🐬", shark: "🦈", octopus: "🐙", crab: "🦀"
```

```
lobster: "🦞", shrimp: "🦐", squirrel: "🐿️", hedgehog: "🦔", fox: "🦊", wolf:
panda: "🐼", giraffe: "🦒", camel: "🐪", zebra: "🦓", deer: "🦌", kangaroo: "
koala: "🐨", flamingo: "🦩", swan: "🦢", rooster: "🐓", dragon: "🐲", unicorn:
ant: "🐜", mosquito: "🦟", scorpion: "🦂", snail: "🐌", worm: "🪱", paw: "🐾",
mouse: "🐭",

// Food & drinks
apple: "🍎", banana: "🍌", lemon: "🍋", grape: "🍇", strawberry: "🍓",
watermelon: "🍉", pineapple: "🍍", mango: "🥭", peach: "🍑", cherry: "🍒", pea
coconut: "🥥", kiwi: "🥝", avocado: "🥑", tomato: "🍅", potato: "🥔", carrot:
corn: "🌽", pepper: "🌶️", cucumber: "🥒", broccoli: "🥦", garlic: "🧄", onion:
mushroom: "🍄", peanut: "🥜", chestnut: "🌰", bread: "🍞", baguette: "🥖", cro
pretzel: "🥨", bagel: "🥯", pancake: "🥞", waffle: "🧇", cheese: "🧀", egg: "
bacon: "🥓", meat: "🍖", steak: "🥩", sausage: "🌭", hamburger: "🍔",
burger: "🍔", fries: "🍟", pizza: "🍕", taco: "🌮", burrito: "🌯", sandwich: "
salad: "🥗", soup: "🍲", noodle: "🍜", noodles: "🍜", spaghetti: "🍝", pasta:
rice: "🍚", curry: "🍛", sushi: "🍣", dumpling: "🥟", popcorn: "🍿", butter: "
salt: "🧂", honey: "🍯", milk: "🥛", coffee: "☕", tea: "🍵", juice: "🧃",
beer: "🍺", wine: "🍷", cocktail: "🍸", whisky: "🥃", whiskey: "🥃", champagne
bottle: "🍾", cup: "☕", glass: "🥃", water: "💧", chocolate: "🍫",
candy: "🍬", lollipop: "🍭", cake: "🍰", cookie: "🍪", donut: "🍩",
doughnut: "🍩", pie: "🥧", icecream: "🍦", "ice cream": "🍦", pudding: "🍮",
cooking: "🍳", chef: "👨‍🍳", restaurant: "🍽️", food: "🍽️",
fork: "🍴", spoon: "🥄", plate: "🍽️", bowl: "🥣",

// Body parts
eye: "👁️", eyes: "👀", ear: "👂", nose: "👃", mouth: "👄", lips: "👄", tongue:
tooth: "🦷", teeth: "🦷", hand: "🖐️", hands: "👐", finger: "👆", thumb: "👍",
fist: "✊", clap: "👏", muscle: "💪", arm: "💪", leg: "🦵", foot: "🦶",
feet: "🦶", brain: "🧠", lung: "🫁", lungs: "🫁", bone: "🦴",
hair: "👱", skin: "🖐️", body: "🧍", face: "😀", head: "🗣️",

// People
child: "🧒", boy: "👦", girl: "👧", man: "👨", woman: "👩", person: "👤",
family: "👪", couple: "💑", friend: "🤝", friends: "👫", people: "👥", crowd:
king: "🤴", queen: "👸", prince: "🤴", princess: "👸", doctor: "👨‍⚕️", nurse: "
teacher: "👩‍🏫", student: "👨‍🎓", police: "👮", soldier: "🪖", farmer: "🧑‍🌾",
cook: "👨‍🍳", artist: "👨‍🎨", singer: "🧟", scientist: "👨‍🔬", judge: "👨‍⚖️",
pilot: "👨‍✈️", astronaut: "👨‍🚀", priest: "👨‍", baby: "👶",

// Emotions / actions
happy: "😊", sad: "😢", angry: "😡", laugh: "😂",
sleep: "😴", sleeping: "😴", tired: "😩", surprised: "😲", surprise: "🎉",
scared: "😨", fear: "😨", afraid: "😨", sick: "🤒", thinking: "🤔", think: "🤔
crazy: "🤪", confused: "😕", bored: "😑", proud: "😌", shy: "😳",
party: "🎉", celebration: "🎉", celebrate: "🎉", kiss: "💋", hug: "🤗", thanks
pray: "🙏", prayer: "🙏", okay: "👌", ok: "👌", good: "👍", bad: "👎",
```

```
yes: "✅", no: "❌", run: "🏃", running: "🏃", walk: "🚶", walking: "🚶", wave
jump: "🤸", swim: "🏊", swimming: "🏊", dance: "💃", dancing: "💃", sing: "🎤"
singing: "🎤", read: "📖", reading: "📖", write: "✍️", writing: "✍️", work: "💼
working: "💼", study: "📚", studying: "📚", learn: "📚", learning: "📚", play:
playing: "🎮", listen: "👂", hear: "👂", speak: "🗣️", talk: "💬", talking: "💬
eat: "🍽️", eating: "🍽️", drink: "🥤", drinking: "🥤", cry: "😭", smile: "🙂",

// Nature & weather
sun: "☀️", moon: "🌙", star: "⭐", stars: "✨", earth: "🌍", world: "🌍", plan
cloud: "☁️", clouds: "☁️", rain: "🌧️", raining: "🌧️", snow: "❄️", snowing: "🌨
wind: "💨", windy: "💨", storm: "🌩️", thunder: "⚡", lightning: "⚡", rainbow:
fog: "🌫️", sky: "🌌", sunrise: "🌅", sunset: "🌇", night: "🌃", day: "☀️",
morning: "🌄", evening: "🌆", winter: "❄️", summer: "☀️", spring: "🌷", autumn
fall: "🍂", tree: "🌳", trees: "🌳", forest: "🌲", flower: "🌸", flowers: "💐"
rose: "🌹", tulip: "🌷", grass: "🌱", leaf: "🍃", leaves: "🍃", plant: "🌱",
seed: "🌱", mountain: "⛰️", volcano: "🌋", desert: "🏜️", island: "🏝️", beach:
sea: "🌊", ocean: "🌊", river: "🏞️", lake: "🏞️", waterfall: "💦",
ice: "🧊", earthquake: "💥",

// Space
comet: "☄️", rocket: "🚀", satellite: "🛰️", ufo: "🛸",
galaxy: "🌌", space: "🌌", alien: "👽", telescope: "🔭",

// Transport
car: "🚗", taxi: "🚕", bus: "🚌", truck: "🚚", van: "🚐", motorcycle: "🏍️",
bike: "🚲", bicycle: "🚲", scooter: "🛴", train: "🚆", subway: "🚇", tram: "🚊
airplane: "✈️", plane: "✈️", flight: "✈️", helicopter: "🚁", boat: "⛵", ship:
yacht: "🛥️", anchor: "⚓", sail: "⛵", traffic: "🚦", parking: "🅿️", fuel: "⛽
gasoline: "⛽", road: "🛣️", bridge: "🌉", tunnel: "🚇",

// Buildings & places
house: "🏠", home: "🏠", building: "🏢", office: "🏢", school: "🏫", hospital:
bank: "🏦", hotel: "🏨", church: "⛪", mosque: "🕌", temple: "🛕", synagogue:
shop: "🏪", store: "🏬", market: "🛒", factory: "🏭", castle: "🏰", museum: "🏛
library: "📚", stadium: "🏟️", farm: "🚜", garden: "🌳", park: "🏞️", city: "🏙
village: "🏘️", town: "🏘️", country: "🌍", room: "🛏️", kitchen: "🍳", bathroom:
bed: "🛏️", bedroom: "🛏️", door: "🚪", window: "🪟", roof: "🏠",

// Objects & tools
phone: "📱", smartphone: "📱", mobile: "📱", computer: "💻", laptop: "💻", tab
keyboard: "⌨️", screen: "🖥️", television: "📺", tv: "📺", radio: "📻",
camera: "📷", microphone: "🎤", headphones: "🎧", speaker: "🔊",
printer: "🖨️", battery: "🔋", plug: "🔌", lamp: "💡", light: "💡", candle: "🕯
clock: "🕐", watch: "⌚", time: "⏰", alarm: "⏰", calendar: "📅", date: "📅",
hourglass: "⏳", money: "💰", cash: "💵", coin: "🪙", credit: "💳", card: "💳"
bag: "👜", backpack: "🎒", briefcase: "💼", luggage: "💼", suitcase: "💼",
key: "🔑", lock: "🔒", padlock: "🔒", chain: "⛓️", rope: "🪢", hammer: "🔨",
```

```
screwdriver: "🪛", wrench: "🔧", saw: "🪚", scissors: "✂️", knife: "🔪", sword
gun: "🔫", bomb: "💣", shield: "🛡️", magnet: "🧲", thermometer: "🌡️", microsco
pencil: "✏️", paintbrush: "🖌️", crayon: "🖍️", document: "📄",
book: "📖", books: "📚", notebook: "📓", newspaper: "📰", envelope: "✉️",
mail: "📫", email: "📧", message: "💬", pin: "📌", paperclip: "📎",
ruler: "📏", calculator: "🧮", abacus: "🧮", glasses: "👓", sunglasses: "🕶️",
pen: "🖊️", paper: "📄", note: "📝", letter: "✉️", video: "📹",

// Clothes & accessories
shirt: "👕", "t-shirt": "👕", tshirt: "👕", jeans: "👖", pants: "👖", trousers
dress: "👗", skirt: "👗", suit: "👔", tie: "👔", coat: "🧥", jacket: "🧥",
shoe: "👞", shoes: "👞", boot: "👢", sandal: "👡", socks: "🧦", hat: "🎩",
cap: "🧢", crown: "👑", helmet: "⛑️", ring: "💍", necklace: "📿", gem: "💎",
diamond: "💎", umbrella: "☂️",

// Sports & games
ball: "⚽", football: "⚽", soccer: "⚽", basketball: "🏀", tennis: "🎾", base
volleyball: "🏐", rugby: "🏉", golf: "⛳", bowling: "🎳", boxing: "🥊", chess:
game: "🎮", trophy: "🏆", medal: "🎖️", target: "🎯", dice: "🎲",
puzzle: "🧩", balloon: "🎈", gift: "🎁", present: "🎁", flag: "🚩", drum: "🥁"
guitar: "🎸", piano: "🎹", violin: "🎻", trumpet: "🎺", saxophone: "🎷",
music: "🎵", song: "🎵", art: "🎨", painting: "🖼️",

// Symbols & abstract
heart: "❤️", love: "❤️", broken: "💔", fire: "🔥", hot: "🔥", warm: "🔥", colc
cool: "😎", new: "🆕", free: "🆓", up: "⬆️", down: "⬇️", left: "⬅️", right: "➡
check: "✅", cross: "❌", warning: "⚠️", danger: "⚠️", stop: "🛑",
prohibited: "🚫", forbidden: "🚫", recycle: "♻️", peace: "☮️",
red: "🔴", green: "🟢", blue: "🔵", yellow: "🟡", orange: "🟠", purple: "🟣",
brown: "🟤", black: "⚫", white: "⚪", pink: "🌸", gold: "🥇", silver: "🥈",

// Numbers
zero: "0️⃣", one: "1️⃣", two: "2️⃣", three: "3️⃣", four: "4️⃣", five: "5️⃣",
six: "6️⃣", seven: "7️⃣", eight: "8️⃣", nine: "9️⃣", ten: "🔟",
hundred: "💯", million: "💰",

// Misc
birthday: "🎂", christmas: "🎄", halloween: "🎃", easter: "🐣", wedding: "💒",
death: "💀", devil: "😈", angel: "👼", ghost: "👻", monster: "👹", magic: "✨"
injection: "💉", pill: "💊", medicine: "💊", health: "💗",
education: "🎓", graduation: "🎓", university: "🏛️", college: "🎓",
language: "🗣️", word: "💬", sentence: "📝", alphabet: "🔤",
number: "🔢", count: "🔢", math: "➗", science: "🔬", religion: "🕊️", god: "🙏
bible: "📖", quran: "📖", spirit: "👻", soul: "👻",
dream: "💭", idea: "💡", question: "❓", answer: "💡", problem: "⚠️", solution
weight: "⚖️", balance: "⚖️", justice: "⚖️", truth: "💯", lie: "🥴",
sport: "⚽", exercise: "🏋️", gym: "🏋️", race: "🏁", winner: "🏆",
```

```
  desk: "🪑", chair: "🪑", table: "🪑", sofa: "🛋️", couch: "🛋️",
  carpet: "🟫", floor: "🧹", bathtub: "🛁", shower: "🚿", soap: "🧼", towel: "🧖
  brush: "🪥", toothbrush: "🪥",
};

function findEmoji(englishWord: string | undefined): string | undefined {
  if (!englishWord) return undefined;
  const cleaned = englishWord.toLowerCase().replace(/[^a-z\s'-]/g, "").trim();
  if (!cleaned) return undefined;
  if (EMOJI_MAP[cleaned]) return EMOJI_MAP[cleaned];
  // Try each token
  const tokens = cleaned.split(/\s+/);
  for (const tok of tokens) {
    if (EMOJI_MAP[tok]) return EMOJI_MAP[tok];
  }
  // Try simple stem (drop trailing s, ed, ing)
  for (const tok of tokens) {
    const stems = [
      tok.replace(/ies$/, "y"),
      tok.replace(/s$/, ""),
      tok.replace(/ed$/, ""),
      tok.replace(/ing$/, ""),
      tok.replace(/ing$/, "e"),
    ];
    for (const s of stems) {
      if (s !== tok && EMOJI_MAP[s]) return EMOJI_MAP[s];
    }
  }
  return undefined;
}

// ─── Translation API ─────────────────────────────────────────────────────

const sentenceCache = new Map<string, string>();
const wordCache = new Map<string, WordApiResult>();

type WordApiResult = {
  translation: string | null;
  pos: string | null;          // noun / verb / etc.
  altMeanings: string[];        // back-dictionary alt translations
};

function buildGoogleUrl(text: string, sl: string, tl: string, dts: string[]): str
  const dtParams = dts.map(d => `&dt=${d}`).join("");
  return `https://translate.googleapis.com/translate_a/single?client=gtx&sl=${sl}
}
```

```
async function googleTranslateSentence(text: string, sl: string, tl: string): Pro
  const key = `s:${sl}>${tl}:${text}`;
  if (sentenceCache.has(key)) return sentenceCache.get(key)!;
  try {
    const res = await fetch(buildGoogleUrl(text, sl, tl, ["t"]));
    if (!res.ok) return null;
    const data = await res.json();
    if (!Array.isArray(data) || !Array.isArray(data[0])) return null;
    const parts = (data[0] as unknown[])
      .map(seg => Array.isArray(seg) ? (seg[0] as string) : "")
      .filter(Boolean);
    const out = parts.join("").trim();
    if (out) sentenceCache.set(key, out);
    return out || null;
  } catch { return null; }
}

async function googleTranslateWord(text: string, sl: string, tl: string): Promise
  const key = `w:${sl}>${tl}:${text.toLowerCase()}`;
  if (wordCache.has(key)) return wordCache.get(key)!;
  const empty: WordApiResult = { translation: null, pos: null, altMeanings: [] };
  try {
    const res = await fetch(buildGoogleUrl(text, sl, tl, ["t", "bd"]));
    if (!res.ok) return empty;
    const data = await res.json();
    if (!Array.isArray(data)) return empty;
    let translation: string | null = null;
    if (Array.isArray(data[0])) {
      const parts = (data[0] as unknown[])
        .map(seg => Array.isArray(seg) ? (seg[0] as string) : "")
        .filter(Boolean);
      translation = parts.join("").trim() || null;
    }
    let pos: string | null = null;
    const altMeanings: string[] = [];
    if (Array.isArray(data[1]) && data[1].length > 0) {
      const firstEntry = data[1][0];
      if (Array.isArray(firstEntry) && typeof firstEntry[0] === "string") {
        pos = firstEntry[0].toLowerCase();
      }
      // Collect alt meanings from all pos categories
      for (const entry of data[1] as unknown[]) {
        if (!Array.isArray(entry)) continue;
        const terms = entry[1];
        if (Array.isArray(terms)) {
          for (const t of terms) {
            if (typeof t === "string" && t && t !== translation && !altMeanings.i
```

```
              altMeanings.push(t);
              if (altMeanings.length >= 6) break;
            }
          }
        }
        if (altMeanings.length >= 6) break;
      }
    }
    const result: WordApiResult = { translation, pos, altMeanings };
    wordCache.set(key, result);
    return result;
  } catch { return empty; }
}

// Synonyms in source language (use Google synsets via dt=ss)
const synonymCache = new Map<string, string | null>();

async function googleSynonym(word: string, sl: string): Promise<string | null> {
  const key = `syn:${sl}:${word.toLowerCase()}`;
  if (synonymCache.has(key)) return synonymCache.get(key)!;
  try {
    // dt=ss gives synonym groups; we ask tl=en (any) but keep sl explicit
    const res = await fetch(buildGoogleUrl(word, sl, "en", ["ss", "t"]));
    if (!res.ok) { synonymCache.set(key, null); return null; }
    const data = await res.json();
    if (!Array.isArray(data)) { synonymCache.set(key, null); return null; }
    // Synonyms appear in data[11]
    const synsets = data[11];
    if (Array.isArray(synsets)) {
      for (const group of synsets as unknown[]) {
        if (!Array.isArray(group)) continue;
        const arr = group[1];
        if (Array.isArray(arr)) {
          for (const sub of arr as unknown[]) {
            if (Array.isArray(sub)) {
              const list = sub[0];
              if (Array.isArray(list)) {
                for (const s of list) {
                  if (typeof s === "string" && s && s.toLowerCase() !== word.toLo
                    synonymCache.set(key, s);
                    return s;
                  }
                }
              }
            } else if (typeof sub === "string" && sub && sub.toLowerCase() !== wo
              synonymCache.set(key, sub);
              return sub;
```

```
            }
          }
        }
      }
    }
    synonymCache.set(key, null);
    return null;
  } catch { return null; }
}

// Pronunciation (romanization of source word, useful for non-Latin scripts)
const pronCache = new Map<string, string | null>();

async function googlePronunciation(word: string, sl: string): Promise<string | nu
  const key = `rm:${sl}:${word.toLowerCase()}`;
  if (pronCache.has(key)) return pronCache.get(key)!;
  try {
    const res = await fetch(buildGoogleUrl(word, sl, "en", ["t", "rm"]));
    if (!res.ok) { pronCache.set(key, null); return null; }
    const data = await res.json();
    if (!Array.isArray(data) || !Array.isArray(data[0])) { pronCache.set(key, nul
    let srcTranslit: string | null = null;
    for (const seg of data[0] as unknown[]) {
      if (!Array.isArray(seg)) continue;
      // src translit usually at index 3 when dt=rm requested
      if (typeof seg[3] === "string" && seg[3]) {
        srcTranslit = (srcTranslit || "") + seg[3];
      }
    }
    if (srcTranslit) srcTranslit = srcTranslit.trim();
    const result = srcTranslit || null;
    pronCache.set(key, result);
    return result;
  } catch { return null; }
}

// Detect language via Google
async function detectLangViaAPI(text: string): Promise<string | null> {
  try {
    const url = buildGoogleUrl(text.slice(0, 200), "auto", "en", ["t", "ld"]);
    const res = await fetch(url);
    if (!res.ok) return null;
    const data = await res.json();
    let detected: string | null = null;
    if (Array.isArray(data) && typeof data[2] === "string") detected = data[2];
    else if (Array.isArray(data) && typeof data[8]?.[3]?.[0] === "string") detect
    if (!detected) return null;
```

```
    if (detected === "iw") detected = "he";
    return LANGUAGES[detected] ? detected : null;
  } catch { return null; }
}

function detectLangFast(text: string): string {
  const arChars = (text.match(/[\u0600-\u06FF]/g) || []).length;
  const cnChars = (text.match(/[\u4E00-\u9FFF]/g) || []).length;
  const jpChars = (text.match(/[\u3040-\u30FF]/g) || []).length;
  const krChars = (text.match(/[\uAC00-\uD7AF]/g) || []).length;
  const heChars = (text.match(/[\u0590-\u05FF]/g) || []).length;
  const cyrChars = (text.match(/[\u0400-\u04FF]/g) || []).length;
  const latinChars = (text.match(/[a-zA-Z]/g) || []).length;
  const counts = [
    { c: arChars, l: "ar" },
    { c: cnChars, l: "zh-CN" },
    { c: jpChars, l: "ja" },
    { c: krChars, l: "ko" },
    { c: heChars, l: "he" },
    { c: cyrChars, l: "ru" },
    { c: latinChars, l: "en" },
  ];
  counts.sort((a, b) => b.c - a.c);
  return counts[0].c > 0 ? counts[0].l : "en";
}

// Example sentence via Google (dt=ex returns examples)
const exampleCache = new Map<string, string | null>();
async function googleExample(word: string, sl: string): Promise<string | null> {
  const key = `ex:${sl}:${word.toLowerCase()}`;
  if (exampleCache.has(key)) return exampleCache.get(key)!;
  try {
    const res = await fetch(buildGoogleUrl(word, sl, "en", ["t", "ex"]));
    if (!res.ok) { exampleCache.set(key, null); return null; }
    const data = await res.json();
    let example: string | null = null;
    if (Array.isArray(data) && Array.isArray(data[13])) {
      const ex = data[13];
      if (Array.isArray(ex[0]) && Array.isArray(ex[0][0]) && typeof ex[0][0][0] =
        example = ex[0][0][0].replace(/<[^>]+>/g, "").trim();
      }
    }
    exampleCache.set(key, example);
    return example;
  } catch { return null; }
}
```

```
// Normalize for answer comparison
function normalizeAnswer(s: string, lang: string, ignoreDiacritics: boolean): str
  let out = (s || "").trim().toLowerCase();
  // Strip Arabic diacritics
  if (lang === "ar" || ignoreDiacritics) {
    out = out.replace(/[\u064B-\u0652\u0670\u0640]/g, "");
  }
  // Normalize Arabic alef variants
  if (lang === "ar") {
    out = out.replace(/[ٱأآإ]/g, "ا").replace(/ى/g, "ي").replace(/ة/g, "ه");
  }
  // Strip punctuation/quotes
  out = out.replace(/[.,!?;:'"()\[\]{}«»""'']/g, "").replace(/\s+/g, " ").trim();
  return out;
}

function answersMatch(user: string, correct: string, lang: string, ignoreDiacriti
  if (!user || !correct) return false;
  const u = normalizeAnswer(user, lang, ignoreDiacritics);
  // Correct answer may be comma-separated alternatives
  const alts = correct.split(/[,،/;|]/).map(a => normalizeAnswer(a, lang, ignoreD
  return alts.includes(u);
}

// Lightweight audio cues using WebAudio (no external assets)
let _audioCtx: AudioContext | null = null;
function getAudioCtx(): AudioContext | null {
  try {
    if (!_audioCtx) _audioCtx = new (window.AudioContext || (window as unknown as
    return _audioCtx;
  } catch { return null; }
}
function playTone(freqs: number[], duration = 0.18, type: OscillatorType = "sine"
  const ctx = getAudioCtx();
  if (!ctx) return;
  freqs.forEach((freq, i) => {
    const osc = ctx.createOscillator();
    const gain = ctx.createGain();
    osc.type = type;
    osc.frequency.value = freq;
    const start = ctx.currentTime + i * (duration * 0.7);
    gain.gain.setValueAtTime(0, start);
    gain.gain.linearRampToValueAtTime(0.18, start + 0.01);
    gain.gain.exponentialRampToValueAtTime(0.0001, start + duration);
    osc.connect(gain).connect(ctx.destination);
    osc.start(start);
    osc.stop(start + duration);
```

```
  });
}
function playCorrectSound() { playTone([523.25, 659.25, 783.99], 0.16); }
function playWrongSound() { playTone([220, 196], 0.22, "triangle"); }
function playFinishSound() { playTone([523.25, 659.25, 783.99, 1046.5], 0.22); }

// ─── Word translation orchestrator ──────────────────────────────────────

type WordDetail = {
  word: string;
  srcLang: string;
  pos: string | null;
  synonym: string | null;
  pronunciation: string | null;
  emoji: string | null;
  translations: Record<string, string>;       // lang -> translation (Google)
  altMeanings: Record<string, string[]>;      // lang -> alt meanings
  // ─── User extensions ───
  userTranslations?: Record<string, string>;  // user-overridden translations
  userNote?: string;                          // free-form note
  exampleSentence?: string;                   // source-lang example
  exampleTranslations?: Record<string, string>; // cached sentence translations p
  needsReview?: boolean;
  errorCount?: number;
  successCount?: number;
  savedAt?: number;
};

// Effective translation = user override OR Google
function effectiveTr(w: WordDetail, tl: string): string {
  return (w.userTranslations?.[tl] ?? w.translations[tl] ?? "").toString();
}

async function fetchWordDetail(word: string, srcLang: string, targetLangs: string
  const cleanWord = word.trim();
  // 1) Translate to each target lang via word endpoint
  const translations: Record<string, string> = {};
  const altMeanings: Record<string, string[]> = {};
  let pos: string | null = null;

  await Promise.all(targetLangs.map(async (tl) => {
    if (tl === srcLang) return;
    const r = await googleTranslateWord(cleanWord, srcLang, tl);
    if (r.translation) translations[tl] = r.translation;
    if (r.altMeanings.length) altMeanings[tl] = r.altMeanings;
    if (!pos && r.pos) pos = r.pos;
  }));
```

```
// 2) Synonym in source language
const synonym = await googleSynonym(cleanWord, srcLang);

// 3) Pronunciation (only useful for non-Latin source)
let pronunciation: string | null = null;
const isLatin = /^[a-zA-Z\s'-]+$/.test(cleanWord);
if (!isLatin) {
  pronunciation = await googlePronunciation(cleanWord, srcLang);
}

// 4) Emoji lookup
let emoji: string | null = null;
let englishKey: string | undefined;
if (srcLang === "en") {
  englishKey = cleanWord;
} else if (translations.en) {
  englishKey = translations.en;
} else {
  // need english for emoji lookup
  const en = await googleTranslateSentence(cleanWord, srcLang, "en");
  englishKey = en || undefined;
}
emoji = findEmoji(englishKey) || null;

// 5) Example sentence (best-effort, English fallback for non-Latin words)
let exampleSentence: string | undefined;
try {
  const ex = await googleExample(cleanWord, srcLang);
  if (ex) exampleSentence = ex;
  else if (englishKey && srcLang !== "en") {
    const exEn = await googleExample(englishKey, "en");
    if (exEn) exampleSentence = exEn;
  }
} catch { /* ignore */ }

return {
  word: cleanWord,
  srcLang,
  pos,
  synonym,
  pronunciation,
  emoji,
  translations,
  altMeanings,
  exampleSentence,
  savedAt: Date.now(),
```

```
    errorCount: 0,
    successCount: 0,
  };
}

// ─── Paragraph building ──────────────────────────────────────────────

type Token = { id: number; raw: string; isWord: boolean };
type Paragraph = {
  id: number;
  tokens: Token[];
  translations: Record<string, string>;
  translating: boolean;
  srcLang: string;
  langConfirmed: boolean;
};

function splitOnPeriods(text: string): string {
  return text
    .replace(/(?<=\D)\.([ \t]*\n+|[ \t]+|$)/g, ".\n\n")
    .replace(/\n{3,}/g, "\n\n")
    .trim();
}

const SPLIT_RE = /(\s+|(?<=[.,!?;:"""'')\[\]\u2014\u2013،؟؛])|(?=[.,!?;:"""'')\
const WORD_RE = /[\p{L}\p{M}]+/u;

function buildParagraphs(rawText: string, srcLangChoice: string | "auto"): Paragr
  let gid = 0;
  return splitOnPeriods(rawText).split(/\n{2,}/).map(p => p.trim()).filter(Boolea
    const detected = srcLangChoice === "auto" ? detectLangFast(para) : srcLangCho
    return {
      id: pi,
      tokens: para.split(SPLIT_RE).filter(Boolean).map(t => ({
        id: gid++, raw: t, isWord: WORD_RE.test(t)
      })),
      translations: {},
      translating: false,
      srcLang: detected,
      langConfirmed: srcLangChoice !== "auto",
    };
  });
}

// ─── Local storage ──────────────────────────────────────────────

const LS_SAVED = "lengoali_saved_v4";
```

```
const LS_QUIZ_PREFS = "lengoali_quiz_prefs_v1";
const LS_DAILY_GOAL = "lengoali_daily_goal_v1";   // { date: "YYYY-MM-DD", goal:
const LS_STREAK = "lengoali_streak_v1";           // { lastDate: "YYYY-MM-DD", da
const LS_SOUND = "lengoali_sound_v1";

function todayKey(): string { return new Date().toISOString().slice(0, 10); }

type QuizPrefs = { mode: "type" | "choice" | "reverse"; ignoreDiacritics: boolean
function loadQuizPrefs(): QuizPrefs {
  try {
    const raw = localStorage.getItem(LS_QUIZ_PREFS);
    if (raw) return { mode: "type", ignoreDiacritics: true, timer: false, sound:
  } catch { /* ignore */ }
  return { mode: "type", ignoreDiacritics: true, timer: false, sound: true };
}
function saveQuizPrefs(p: QuizPrefs) {
  try { localStorage.setItem(LS_QUIZ_PREFS, JSON.stringify(p)); } catch { /* igno
}

type DailyGoal = { date: string; goal: number; learned: number };
function loadDailyGoal(): DailyGoal {
  try {
    const raw = localStorage.getItem(LS_DAILY_GOAL);
    if (raw) {
      const g = JSON.parse(raw);
      if (g.date === todayKey()) return g;
    }
  } catch { /* ignore */ }
  return { date: todayKey(), goal: 10, learned: 0 };
}
function saveDailyGoal(g: DailyGoal) {
  try { localStorage.setItem(LS_DAILY_GOAL, JSON.stringify(g)); } catch { /* igno
}

type Streak = { lastDate: string; days: number };
function loadStreak(): Streak {
  try {
    const raw = localStorage.getItem(LS_STREAK);
    if (raw) return JSON.parse(raw);
  } catch { /* ignore */ }
  return { lastDate: "", days: 0 };
}
function bumpStreak(): Streak {
  const cur = loadStreak();
  const today = todayKey();
  if (cur.lastDate === today) return cur;
  const yesterday = new Date(Date.now() - 86400000).toISOString().slice(0, 10);
```

```
  const next: Streak = {
    lastDate: today,
    days: cur.lastDate === yesterday ? cur.days + 1 : 1,
  };
  try { localStorage.setItem(LS_STREAK, JSON.stringify(next)); } catch { /* ignor
  return next;
}
const LS_THEME = "lengoali_theme";
const LS_UI_LANG = "lengoali_ui_lang";
const LS_SOURCE = "lengoali_source_lang";
const LS_TARGETS = "lengoali_target_langs";

function loadSaved(): WordDetail[] {
  try {
    let raw = localStorage.getItem(LS_SAVED);
    if (!raw) {
      // migrate from v3 if present
      raw = localStorage.getItem("lengoali_saved_v3");
      if (raw) {
        try { localStorage.setItem(LS_SAVED, raw); } catch { /* ignore */ }
      }
    }
    const arr: WordDetail[] = raw ? JSON.parse(raw) : [];
    return arr.map((w, i) => ({
      ...w,
      savedAt: w.savedAt || Date.now() - (arr.length - i) * 1000,
      errorCount: w.errorCount || 0,
      successCount: w.successCount || 0,
    }));
  } catch { return []; }
}
function persistSaved(arr: WordDetail[]) {
  try { localStorage.setItem(LS_SAVED, JSON.stringify(arr)); } catch { /* ignore
}

// ─── Reusable language picker ──────────────────────────────────────

function LangSelect({
  value, onChange, options, placeholder, t, allowAuto, autoLabel, onRemove,
}: {
  value: string | "auto";
  onChange: (v: string) => void;
  options: string[];
  placeholder?: string;
  t: Theme;
  allowAuto?: boolean;
  autoLabel?: string;
```

```
  onRemove?: () => void;
}) {
  const [open, setOpen] = useState(false);
  const ref = useRef<HTMLDivElement | null>(null);
  useEffect(() => {
    function onDoc(e: MouseEvent) {
      if (ref.current && !ref.current.contains(e.target as Node)) setOpen(false);
    }
    document.addEventListener("mousedown", onDoc);
    return () => document.removeEventListener("mousedown", onDoc);
  }, []);
  const display = value === "auto"
    ? (autoLabel || "Auto")
    : `${getLang(value).flag} ${getLang(value).name}`;
  return (
    <div ref={ref} style={{ position: "relative", flex: 1, minWidth: 0 }}>
      <div style={{ display: "flex", gap: 6, alignItems: "stretch" }}>
        <button
          onClick={() => setOpen(o => !o)}
          style={{
            flex: 1,
            display: "flex", alignItems: "center", justifyContent: "space-between
            gap: 6, padding: "10px 12px", borderRadius: 10,
            border: `1px solid ${t.inputBorder}`, background: t.inputBg, color: t
            fontSize: 14, cursor: "pointer", textAlign: "left", minWidth: 0,
          }}
        >
          <span style={{ overflow: "hidden", textOverflow: "ellipsis", whiteSpace
            {value ? display : (placeholder || "Select…")}
          </span>
          <ChevronDown size={14} style={{ flexShrink: 0, opacity: 0.6, transform:
        </button>
        {onRemove && (
          <button
            onClick={onRemove}
            title="Remove"
            style={{
              padding: "0 10px", borderRadius: 10, border: `1px solid ${t.inputBo
              background: t.inputBg, color: "#ef4444", cursor: "pointer", display
            }}
          >
            <X size={14} />
          </button>
        )}
      </div>
      {open && (
        <div style={{
```

```
          position: "absolute", top: "calc(100% + 4px)", left: 0, right: 0, zInde
          background: t.card, border: `1px solid ${t.border}`, borderRadius: 10,
          maxHeight: 280, overflowY: "auto", boxShadow: "0 8px 30px rgba(0,0,0,.2
        }}>
          {allowAuto && (
            <button
              onClick={() => { onChange("auto"); setOpen(false); }}
              style={{
                display: "block", width: "100%", textAlign: "left", padding: "10p
                background: value === "auto" ? `${t.border}55` : "transparent",
                border: "none", color: t.text, fontSize: 14, cursor: "pointer",
              }}
            >
              🌐 {autoLabel || "Auto-detect"}
            </button>
          )}
          {options.map(code => {
            const lang = getLang(code);
            return (
              <button
                key={code}
                onClick={() => { onChange(code); setOpen(false); }}
                style={{
                  display: "block", width: "100%", textAlign: "left", padding: "1
                  background: value === code ? `${t.border}55` : "transparent",
                  border: "none", color: t.text, fontSize: 14, cursor: "pointer",
                }}
              >
                <span style={{ marginRight: 8 }}>{lang.flag}</span>
                {lang.name} <span style={{ color: t.textDim, fontSize: 12 }}>· {l
              </button>
            );
          })}
        </div>
      )}
    </div>
  );
}

// ─── UI language switcher (compact in header) ──────────────────────────

function UiLangSwitcher({ ui, setUi, t }: { ui: UiLang; setUi: (u: UiLang) => voi
  const [open, setOpen] = useState(false);
  const ref = useRef<HTMLDivElement | null>(null);
  useEffect(() => {
    function onDoc(e: MouseEvent) {
      if (ref.current && !ref.current.contains(e.target as Node)) setOpen(false);
```

```
    }
    document.addEventListener("mousedown", onDoc);
    return () => document.removeEventListener("mousedown", onDoc);
  }, []);
  return (
    <div ref={ref} style={{ position: "relative" }}>
      <button
        onClick={() => setOpen(o => !o)}
        title="UI language"
        style={{
          display: "flex", alignItems: "center", gap: 4,
          width: "auto", height: 32, padding: "0 8px", borderRadius: 16,
          border: `1px solid ${t.border}`, background: t.card2, color: t.textMute
          cursor: "pointer", fontSize: 12, flexShrink: 0,
        }}
      >
        <Globe size={13} />
        <span>{ui.toUpperCase()}</span>
      </button>
      {open && (
        <div style={{
          position: "absolute", top: "calc(100% + 4px)", right: 0, zIndex: 50,
          background: t.card, border: `1px solid ${t.border}`, borderRadius: 10,
          minWidth: 140, boxShadow: "0 8px 30px rgba(0,0,0,.2)",
        }}>
          {UI_LANGS.map(u => (
            <button
              key={u}
              onClick={() => { setUi(u); setOpen(false); }}
              style={{
                display: "block", width: "100%", textAlign: "left", padding: "8px
                background: ui === u ? `${t.border}55` : "transparent",
                border: "none", color: t.text, fontSize: 13, cursor: "pointer",
              }}
            >
              {getLang(u).flag} {getLang(u).nativeName}
            </button>
          ))}
        </div>
      )}
    </div>
  );
}

// ── Flashcard ──────────────────────────────────────────────────────────

function Flashcard({ cards: rawCards, onClose, onLearn, t, s }: { cards: WordDeta
```

```
const [idx, setIdx] = useState(0);
const [revealed, setRevealed] = useState<string | null>(null);
const [order, setOrder] = useState<number[]>(() => rawCards.map((_, i) => i));
useEffect(() => { setOrder(rawCards.map((_, i) => i)); }, [rawCards.length]);
const cards = order.map(i => rawCards[i]).filter(Boolean);
function shuffle() {
  const arr = order.slice();
  for (let i = arr.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [arr[i], arr[j]] = [arr[j], arr[i]];
  }
  setOrder(arr);
  setIdx(0);
  setRevealed(null);
}

useEffect(() => { if (cards.length > 0 && idx >= cards.length) setIdx(cards.len

if (!cards.length) return (
  <div style={{ position: "fixed", inset: 0, zIndex: 60, background: t.overlayF
    <p style={{ color: t.textDim, textAlign: "center", marginTop: 60 }}>{s.noFl
    <button onClick={onClose} style={{ marginTop: 20, padding: "10px 30px", bor
  </div>
);

const safeIdx = Math.min(idx, cards.length - 1);
const card = cards[safeIdx];
const targets = Object.keys(card.translations);
const srcLangInfo = getLang(card.srcLang);

function next() { setRevealed(null); setTimeout(() => setIdx(i => (i + 1) % car
function prev() { setRevealed(null); setTimeout(() => setIdx(i => (i - 1 + card
function handleLearn() {
  const w = card.word;
  if (cards.length === 1) { onLearn(w); onClose(); return; }
  setRevealed(null);
  setIdx(i => Math.min(i, cards.length - 2));
  setTimeout(() => onLearn(w), 0);
}

return (
  <div style={{ position: "fixed", inset: 0, zIndex: 60, background: t.overlayF
    <div style={{ width: "100%", maxWidth: 460, display: "flex", justifyContent
      <span style={{ color: t.textDim, fontSize: 12 }}>{safeIdx + 1} / {cards.l
      <div style={{ display: "flex", gap: 8, alignItems: "center" }}>
        <button onClick={shuffle} title="Shuffle" style={{ background: "#a78bfa
          <Shuffle size={13} /> Shuffle
```

```
        </button>
        <button onClick={onClose} style={{ background: "none", border: "none",
      </div>
    </div>

    <div style={{ width: "100%", maxWidth: 460, marginBottom: 16, background: t
      <span style={{ fontSize: 11, fontWeight: 600, padding: "3px 12px", border
        {srcLangInfo.flag} {srcLangInfo.name}{card.pos ? ` · ${card.pos}` : ""}
      </span>
      <span style={{ fontSize: 38, fontWeight: 700, color: t.text, direction: s
      <SpeakBtn text={card.word} ttsLang={srcLangInfo.ttsCode} size={18} />
      {card.synonym && (
        <span style={{ fontSize: 13, color: t.textDim, fontStyle: "italic", dir
      )}
      {card.pronunciation && (
        <span style={{ fontSize: 13, color: t.textMuted, fontFamily: "monospace
      )}
      <div style={{ display: "flex", flexWrap: "wrap", gap: 6, marginTop: 6, ju
        {targets.map(tl => (
          <button key={tl} onClick={() => setRevealed(revealed === tl ? null :
            style={{ fontSize: 12, padding: "5px 12px", borderRadius: 20, borde
            {getLang(tl).flag} {getLang(tl).name}
          </button>
        ))}
      </div>
      <span style={{ fontSize: 12, color: t.textFaint }}>{s.tapToTranslate}</sp
    </div>

    {revealed && card.translations[revealed] && (
      <div style={{ width: "100%", maxWidth: 460, borderRadius: 16, padding: "1
        <div style={{ display: "flex", alignItems: "center", gap: 12 }}>
          <span style={{ fontSize: 11, color: t.textFaint, minWidth: 60 }}>{get
          <span style={{ fontSize: 22, fontWeight: 700, color: "#60a5fa", direc
            {card.translations[revealed]}
            {card.emoji && <span style={{ marginLeft: 8 }}>{card.emoji}</span>}
          </span>
          <SpeakBtn text={card.translations[revealed]} ttsLang={getLang(reveale
        </div>
      </div>
    )}

    <div style={{ display: "flex", gap: 12, width: "100%", maxWidth: 460 }}>
      <button onClick={prev} style={{ flex: 1, padding: 14, borderRadius: 12, f
      <button onClick={handleLearn} style={{ flex: 1, padding: 14, borderRadius
      <button onClick={next} style={{ flex: 1, padding: 14, borderRadius: 12, f
    </div>
  </div>
```

```
  );
}

// ─── Copy modal ──────────────────────────────────────────────────────────────

function CopyModal({ saved, onClose, t, s }: { saved: WordDetail[]; onClose: () =
  const [copied, setCopied] = useState(false);
  const lines = saved.map(w => {
    const trs = Object.keys(w.translations).map(tl => {
      const emojiPart = w.emoji ? ` ${w.emoji}` : "";
      return `${getLang(tl).name}: ${w.translations[tl]}${emojiPart}`;
    }).join(" | ");
    return `${w.word} — ${trs}`;
  }).join("\n");
  const content = `${s.savedWordsTitle}\n${"=".repeat(30)}\n\n${lines}`;
  function handleCopy() {
    navigator.clipboard?.writeText(content).then(() => { setCopied(true); setTime
  }
  return (
    <div style={{ position: "fixed", inset: 0, zIndex: 70, background: t.overlay,
      <div style={{ background: t.card, borderRadius: 16, border: `1px solid ${t.
        <div style={{ display: "flex", justifyContent: "space-between", alignItem
          <span style={{ fontSize: 15, fontWeight: 700, color: t.text }}>{s.saved
          <button onClick={onClose} style={{ background: "none", border: "none",
        </div>
        <textarea readOnly value={content} style={{ width: "100%", background: t.
        <button onClick={handleCopy} style={{ padding: 12, borderRadius: 10, bord
          {copied ? `✓ ${s.copied}` : `📋 ${s.copyAll}`}
        </button>
      </div>
    </div>
  );
}

// ─── Main App ────────────────────────────────────────────────────────────────

export default function App() {
  const [isDark, setIsDark] = useState<boolean>(() => {
    try { return localStorage.getItem(LS_THEME) !== "light"; } catch { return tru
  });
  const t = isDark ? DARK : LIGHT;
  function toggleTheme() {
    setIsDark(d => {
      const next = !d;
      try { localStorage.setItem(LS_THEME, next ? "dark" : "light"); } catch { /*
      return next;
    });
```

```
}

// UI language
const [ui, setUiState] = useState<UiLang>(() => {
  try {
    const v = localStorage.getItem(LS_UI_LANG);
    if (v && UI_LANGS.includes(v as UiLang)) return v as UiLang;
  } catch { /* ignore */ }
  return "en";
});
const setUi = (u: UiLang) => {
  setUiState(u);
  try { localStorage.setItem(LS_UI_LANG, u); } catch { /* ignore */ }
};
const s = STR[ui];
const uiDir = getLang(ui).dir;

// Source / target languages
const [sourceLang, setSourceLangState] = useState<string | "auto">(() => {
  try { return localStorage.getItem(LS_SOURCE) || "auto"; } catch { return "aut
});
const setSourceLang = (v: string | "auto") => {
  setSourceLangState(v);
  try { localStorage.setItem(LS_SOURCE, v); } catch { /* ignore */ }
};

const [targetLangs, setTargetLangsState] = useState<string[]>(() => {
  try {
    const raw = localStorage.getItem(LS_TARGETS);
    if (raw) {
      const arr = JSON.parse(raw);
      if (Array.isArray(arr) && arr.length >= 2) return arr.filter(x => typeof
    }
  } catch { /* ignore */ }
  return ["ar", "en"];
});
const setTargetLangs = (arr: string[]) => {
  setTargetLangsState(arr);
  try { localStorage.setItem(LS_TARGETS, JSON.stringify(arr)); } catch { /* ign
};

function updateTargetAt(idx: number, code: string) {
  const next = targetLangs.slice();
  next[idx] = code;
  // de-duplicate
  setTargetLangs(Array.from(new Set(next)));
}
```

```
function removeTargetAt(idx: number) {
  const next = targetLangs.slice();
  next.splice(idx, 1);
  if (next.length === 0) next.push("en");
  setTargetLangs(next);
}
function addTarget(code: string) {
  if (targetLangs.includes(code)) return;
  setTargetLangs([...targetLangs, code]);
}

// App state
const [tab, setTab] = useState<"input" | "reader" | "saved" | "quiz">("input");
const [rawText, setRawText] = useState("");
const [paragraphs, setParagraphs] = useState<Paragraph[]>([]);
const [sheet, setSheet] = useState<WordDetail | null>(null);
const [sheetLoading, setSheetLoading] = useState(false);
const [sheetOpen, setSheetOpen] = useState(false);
const [saved, setSaved] = useState<WordDetail[]>(loadSaved);
const [flashMode, setFlashMode] = useState(false);
const [showCopy, setShowCopy] = useState(false);
const [colorize, setColorize] = useState(false);
const [posMap, setPosMap] = useState<Record<string, string>>({});
const [colorizeLoading, setColorizeLoading] = useState(false);
const [readerFontSize, setReaderFontSize] = useState(18);
const [ttsSpeed, setTtsSpeed] = useState<"slow" | "normal" | "fast">("normal");
const activeWordRef = useRef<string | null>(null);

// Quiz / streak / daily goal state
const [quizPrefs, setQuizPrefsState] = useState<QuizPrefs>(loadQuizPrefs);
const setQuizPrefs = (p: QuizPrefs) => { setQuizPrefsState(p); saveQuizPrefs(p)
const [dailyGoal, setDailyGoalState] = useState<DailyGoal>(loadDailyGoal);
const setDailyGoal = (g: DailyGoal) => { setDailyGoalState(g); saveDailyGoal(g)
const [streak, setStreak] = useState<Streak>(loadStreak);
const [editingWord, setEditingWord] = useState<{ word: string; srcLang: string
const [savedSearch, setSavedSearch] = useState("");
const [savedFilter, setSavedFilter] = useState<"all" | "review" | "edited">("al
const [savedSort, setSavedSort] = useState<"newest" | "alpha" | "wrong">("newes

useEffect(() => { persistSaved(saved); }, [saved]);

// Sync TTS speed to global
useEffect(() => {
  globalTtsRate = ttsSpeed === "slow" ? 0.55 : ttsSpeed === "fast" ? 1.5 : 1.0;
}, [ttsSpeed]);

// Update saved word fields (translations, note, example, review flags, counter
```

```
function updateSavedWord(word: string, srcLang: string, patch: Partial<WordDeta
  setSaved(prev => prev.map(w => (w.word === word && w.srcLang === srcLang) ? {
}
function setUserTranslation(word: string, srcLang: string, tl: string, value: s
  setSaved(prev => prev.map(w => {
    if (w.word !== word || w.srcLang !== srcLang) return w;
    const next = { ...(w.userTranslations || {}) };
    const trimmed = value.trim();
    if (trimmed && trimmed !== w.translations[tl]) next[tl] = trimmed;
    else delete next[tl];
    return { ...w, userTranslations: Object.keys(next).length ? next : undefine
  }));
}
async function fetchExampleTranslation(word: string, srcLang: string, tl: strin
  const w = saved.find(x => x.word === word && x.srcLang === srcLang);
  if (!w?.exampleSentence) return;
  if (w.exampleTranslations?.[tl]) return;
  const tr = await googleTranslateSentence(w.exampleSentence, srcLang, tl);
  if (tr) {
    setSaved(prev => prev.map(x => (x.word === word && x.srcLang === srcLang)
      ? { ...x, exampleTranslations: { ...(x.exampleTranslations || {}), [tl]:
      : x));
  }
}

// After a correct quiz answer: bump daily goal + streak
function onQuizCorrect(word: string, srcLang: string) {
  setSaved(prev => prev.map(w => (w.word === word && w.srcLang === srcLang)
    ? { ...w, successCount: (w.successCount || 0) + 1, needsReview: false }
    : w));
  setDailyGoal({ ...dailyGoal, learned: dailyGoal.learned + 1 });
  setStreak(bumpStreak());
}
function onQuizWrong(word: string, srcLang: string) {
  setSaved(prev => prev.map(w => (w.word === word && w.srcLang === srcLang)
    ? { ...w, errorCount: (w.errorCount || 0) + 1, needsReview: true }
    : w));
}

// ─── POS colorization ──────
useEffect(() => {
  if (!colorize || paragraphs.length === 0) return;
  let cancelled = false;
  const tasks: { word: string; lang: string; key: string }[] = [];
  const seen = new Set<string>();
  for (const p of paragraphs) {
    for (const tok of p.tokens) {
```

```
    if (!tok.isWord) continue;
    const cleaned = tok.raw.replace(/[^\p{L}\p{M}'-]/gu, "");
    if (cleaned.length < 2) continue;
    const lower = cleaned.toLowerCase();
    const key = `${lower}|${p.srcLang}`;
    if (seen.has(key) || posMap[key] !== undefined) continue;
    seen.add(key);
    tasks.push({ word: cleaned, lang: p.srcLang, key });
  }
}
if (tasks.length === 0) return;
setColorizeLoading(true);
// Google's bilingual dictionary returns POS most reliably when one side is
// English. So: if source is English, pair with the first non-English target
// (or Arabic). If source is anything else, always pair with English.
const fallbackForEn = targetLangs.find(x => x !== "en" && x !== "auto") || "a
const concurrency = 6;
let cursor = 0;
const updates: Record<string, string> = {};
let flushTimer: ReturnType<typeof setTimeout> | null = null;
const scheduleFlush = () => {
  if (flushTimer) return;
  flushTimer = setTimeout(() => {
    flushTimer = null;
    if (cancelled || Object.keys(updates).length === 0) return;
    setPosMap(prev => ({ ...prev, ...updates }));
    for (const k of Object.keys(updates)) delete updates[k];
  }, 120);
};
async function worker() {
  while (!cancelled && cursor < tasks.length) {
    const i = cursor++;
    const t = tasks[i];
    const primaryTl = t.lang === "en" ? fallbackForEn : "en";
    try {
      let r = await googleTranslateWord(t.word, t.lang, primaryTl);
      // Fallback chain: try the user's first target, then English, then Span
      if (!r.pos) {
        const alt1 = targetLangs.find(x => x !== t.lang && x !== "auto" && x
        if (alt1) r = await googleTranslateWord(t.word, t.lang, alt1);
      }
      if (!r.pos && primaryTl !== "en" && t.lang !== "en") {
        r = await googleTranslateWord(t.word, t.lang, "en");
      }
      updates[t.key] = (r.pos || "").toLowerCase() || "_none_";
    } catch {
      updates[t.key] = "_none_";
```

```
      }
      scheduleFlush();
    }
  }
  Promise.all(Array.from({ length: Math.min(concurrency, tasks.length) }, worke
    .then(() => {
      if (cancelled) return;
      if (Object.keys(updates).length) setPosMap(prev => ({ ...prev, ...updates
      setColorizeLoading(false);
    });
  return () => { cancelled = true; if (flushTimer) clearTimeout(flushTimer); };
}, [colorize, paragraphs, targetLangs]);

// Available languages for picker (exclude source if not auto)
const sourceCode = sourceLang === "auto" ? null : sourceLang;
const availableTargets = useMemo(() => {
  return LANG_CODES.filter(c => c !== sourceCode);
}, [sourceCode]);

// ——— Text actions ————
async function handleRead() {
  const built = buildParagraphs(rawText, sourceLang);
  if (!built.length) return;
  setParagraphs(built);
  setTab("reader");
  if (sourceLang === "auto") {
    built.forEach(async (para) => {
      const text = para.tokens.map(tok => tok.raw).join("");
      const detected = await detectLangViaAPI(text);
      setParagraphs(prev => prev.map(p => p.id === para.id
        ? { ...p, srcLang: detected || p.srcLang, langConfirmed: true }
        : p));
    });
  }
}

async function translateParagraph(pid: number) {
  const para = paragraphs.find(p => p.id === pid);
  if (!para || para.translating) return;
  const text = para.tokens.map(tok => tok.raw).join("");
  setParagraphs(prev => prev.map(p => p.id === pid ? { ...p, translating: true
  const targets = targetLangs.filter(tl => tl !== para.srcLang);
  const results: Record<string, string> = {};
  await Promise.all(targets.map(async (tl) => {
    const tr = await googleTranslateSentence(text, para.srcLang, tl);
    if (tr) results[tl] = tr;
  }));
```

```
  setParagraphs(prev => prev.map(p => p.id === pid ? { ...p, translations: resu
}

const openWord = useCallback(async (raw: string, paraLang: string) => {
  const cleaned = raw.replace(/[^\p{L}\p{M}'-]/gu, "");
  if (!cleaned) return;
  const lower = cleaned.toLowerCase();
  activeWordRef.current = lower;
  setSheet({
    word: cleaned, srcLang: paraLang, pos: null, synonym: null,
    pronunciation: null, emoji: null, translations: {}, altMeanings: {},
  });
  setSheetLoading(true);
  setSheetOpen(true);
  const targets = targetLangs.filter(tl => tl !== paraLang);
  const detail = await fetchWordDetail(cleaned, paraLang, targets);
  if (activeWordRef.current === lower) {
    setSheet(detail);
    setSheetLoading(false);
  }
}, [targetLangs]);

function toggleSave() {
  if (!sheet) return;
  setSaved(prev => {
    const exists = prev.find(w => w.word === sheet.word && w.srcLang === sheet.
    return exists
      ? prev.filter(w => !(w.word === sheet.word && w.srcLang === sheet.srcLang
      : [...prev, sheet];
  });
}
const isSaved = sheet ? saved.some(w => w.word === sheet.word && w.srcLang ===

function deleteSaved(word: string, srcLang: string) {
  setSaved(prev => prev.filter(w => !(w.word === word && w.srcLang === srcLang)
}
function learnWord(word: string) {
  setSaved(prev => prev.filter(w => w.word !== word));
}
function newSession() { setRawText(""); setParagraphs([]); setSheet(null); setS

if (flashMode) return <Flashcard cards={saved} onClose={() => setFlashMode(fals

// ——— Render ————
return (
  <div dir={uiDir} style={{ minHeight: "100dvh", maxWidth: 720, margin: "0 auto
    <style>{`
```

```
@keyframes spin{to{transform:rotate(360deg)}}
@keyframes shake{0%,100%{transform:translateX(0)}20%,60%{transform:translateX(-8p
@keyframes pop{0%{transform:scale(.4);opacity:0}60%{transform:scale(1.15);opacity
@keyframes confetti-fall{0%{transform:translateY(-30px) rotate(0);opacity:1}100%{
@keyframes trophy-bounce{0%{transform:scale(0) rotate(-30deg);opacity:0}50%{trans
@keyframes pulse-ring{0%{transform:scale(.6);opacity:.85}100%{transform:scale(2);
@keyframes flash-correct{0%{background:#22c55e00}30%{background:#22c55e55}100%{ba
@keyframes flash-wrong{0%{background:#ef444400}30%{background:#ef444555}100%{back
*{box-sizing:border-box}
body{margin:0;background:${t.bg}}
textarea:focus,input:focus,button:focus{outline:none}
.shake{animation:shake .45s ease-in-out}
.pop{animation:pop .35s ease-out}
.flash-correct{animation:flash-correct .8s ease-out}
.flash-wrong{animation:flash-wrong .8s ease-out}
`}</style>

      {showCopy && <CopyModal saved={saved} onClose={() => setShowCopy(false)} t=

      {/* Header */}
      <header style={{ position: "sticky", top: 0, zIndex: 10, display: "flex", a
        <span style={{ fontSize: 20, fontWeight: 700, color: "#60a5fa" }}>Lengoal
        <div style={{ flex: 1, display: "flex", gap: 6, alignItems: "center", fle
          {streak.days > 0 && (
            <span title={`${streak.days} ${s.streak}`} style={{ fontSize: 11, pad
              🔥 {streak.days}
            </span>
          )}
          <span title={s.dailyGoal} style={{ fontSize: 11, padding: "3px 8px", bo
            🎯 {Math.min(dailyGoal.learned, dailyGoal.goal)}/{dailyGoal.goal}
          </span>
        </div>
        {tab !== "input" && (
          <button onClick={newSession} style={{ fontSize: 11, padding: "4px 10px"
            ↩ {s.newText}
          </button>
        )}
        {tab === "reader" && (
          <>
            {/* Font size ± */}
            <button
              onClick={() => setReaderFontSize(s => Math.max(12, s - 2))}
              title="تصغير النص"
              style={{ display: "flex", alignItems: "center", justifyContent: "ce
              –
            </button>
            <button
```

```
          onClick={() => setReaderFontSize(s => Math.min(32, s + 2))}
          title="تكبير النص"
          style={{ display: "flex", alignItems: "center", justifyContent: "ce
          +
        </button>
        {/* TTS speed */}
        <button
          onClick={() => setTtsSpeed(sp => sp === "slow" ? "normal" : sp ===
          title={ttsSpeed === "slow" ? "بطيء" : ttsSpeed === "fast" ? "عادي"
          style={{ display: "flex", alignItems: "center", justifyContent: "ce
          {ttsSpeed === "slow" ? "🐢" : ttsSpeed === "fast" ? "🐇" : "🔊"}
        </button>
        {/* Colorize */}
        <button
          onClick={() => { if (!colorizeLoading) setColorize(c => !c); }}
          title={colorize ? s.colorizeOff : s.colorize}
          style={{
            display: "flex", alignItems: "center", justifyContent: "center",
            width: 32, height: 32, borderRadius: "50%",
            border: `1px solid ${colorize ? "#60a5fa" : t.border}`,
            background: colorize ? "#60a5fa22" : t.card2,
            color: colorize ? "#60a5fa" : t.textMuted,
            cursor: colorizeLoading ? "not-allowed" : "pointer",
            padding: 0, flexShrink: 0, position: "relative",
            opacity: colorizeLoading ? 0.7 : 1,
          }}>
          <Palette size={15} />
          {colorize && colorizeLoading && (
            <span style={{
              position: "absolute", top: -2, right: -2, width: 8, height: 8,
              borderRadius: "50%", background: "#60a5fa",
              animation: "spin 0.7s linear infinite",
            }} />
          )}
        </button>
      </>
    )}
    <UiLangSwitcher ui={ui} setUi={setUi} t={t} />
    <button onClick={toggleTheme} title={isDark ? s.light : s.dark}
      style={{ display: "flex", alignItems: "center", justifyContent: "center
      {isDark ? <Sun size={15} /> : <Moon size={15} />}
    </button>
  </header>

  <main style={{ flex: 1, overflowY: "auto", padding: "16px 16px 110px" }}>

    {/* INPUT TAB */}
```

```
{tab === "input" && (
  <div style={{ display: "flex", flexDirection: "column", gap: 16 }}>
    <div style={{ display: "flex", justifyContent: "space-between", align
      <p style={{ fontSize: 13, color: t.textDim, margin: 0, lineHeight:
      {rawText.trim() && (
        <button onClick={newSession}
          style={{ fontSize: 12, padding: "5px 12px", borderRadius: 20, b
          <X size={12} /> {s.newText}
        </button>
      )}
    </div>

    <textarea
      style={{ width: "100%", background: t.inputBg, border: `1px solid $
      value={rawText}
      onChange={e => setRawText(e.target.value)}
      placeholder={s.pastePlaceholder}
      rows={10}
      spellCheck={false}
    />

    {/* Source language */}
    <div style={{ display: "flex", flexDirection: "column", gap: 8 }}>
      <label style={{ fontSize: 12, color: t.textDim, fontWeight: 600 }}>
      <LangSelect
        value={sourceLang}
        onChange={(v) => setSourceLang(v)}
        options={LANG_CODES}
        t={t}
        allowAuto
        autoLabel={s.autoDetect}
      />
    </div>

    {/* Target languages */}
    <div style={{ display: "flex", flexDirection: "column", gap: 8 }}>
      <label style={{ fontSize: 12, color: t.textDim, fontWeight: 600 }}>
      {targetLangs.map((code, idx) => (
        <LangSelect
          key={`${code}-${idx}`}
          value={code}
          onChange={(v) => updateTargetAt(idx, v)}
          options={availableTargets}
          t={t}
          onRemove={targetLangs.length > 1 ? () => removeTargetAt(idx) :
        />
      ))}
```

```
      {/* Add language */}
      <AddLangButton onPick={addTarget} options={availableTargets.filter(
    </div>

    <button onClick={handleRead} disabled={!rawText.trim()} style={{ padd
      {s.readNow} →
    </button>
  </div>
)}

{/* READER TAB */}
{tab === "reader" && (
  <div style={{ display: "flex", flexDirection: "column", gap: 16 }}>
    {paragraphs.map((para, pi) => {
      const targets = targetLangs.filter(tl => tl !== para.srcLang);
      const srcInfo = getLang(para.srcLang);
      return (
        <div key={para.id}>
          <div style={{ marginBottom: 6 }}>
            <span style={{ fontSize: 10, padding: "2px 8px", borderRadius
              {srcInfo.flag} {srcInfo.name}
              {!para.langConfirmed && <span style={{ opacity: 0.5 }}>…</s
            </span>
          </div>
          <div style={{ display: "flex", flexWrap: "wrap", lineHeight: 2.
            {para.tokens.map(tok => {
              if (!tok.isWord) {
                return <span key={tok.id} style={{ fontSize: readerFontSi
              }
              const cleaned = tok.raw.replace(/[^\p{L}\p{M}'-]/gu, "").to
              const posKey = `${cleaned}|${para.srcLang}`;
              const pos = colorize ? posMap[posKey] : undefined;
              const hasPos = pos && pos !== "_none_";
              const wColor = colorize && hasPos ? posColor(pos) : t.text;
              return (
                <span key={tok.id}
                  style={{
                    display: "inline-block", cursor: "pointer",
                    padding: "1px 3px", borderRadius: 4, fontSize: reader
                    userSelect: "none", transition: "color .2s, backgroun
                    color: wColor,
                    fontWeight: colorize && hasPos ? 600 : 400,
                  }}
                  title={colorize && hasPos ? pos : undefined}
                  onClick={() => openWord(tok.raw, para.srcLang)}>
                  {tok.raw}
                </span>
```

```
              );
            })}
          </div>
          <div style={{ background: t.card2, borderRadius: 10, border: `1
            <div style={{ display: "flex", justifyContent: "space-between
              <span style={{ fontSize: 11, color: t.textDim, display: "fl
                {targets.map(tl => <span key={tl}>{getLang(tl).flag}</spa
              </span>
              <div style={{ display: "flex", gap: 6, alignItems: "center"
                <SpeakBtn text={para.tokens.map(tok => tok.raw).join("")}
                <button onClick={() => translateParagraph(para.id)} disab
                  style={{ display: "flex", alignItems: "center", gap: 5,
                  {para.translating
                    ? <><span style={{ display: "inline-block", width: 11
                    : `✦ ${s.translate}`}
                </button>
              </div>
            </div>
            {targets.map(tl => {
              const tr = para.translations[tl];
              const tlInfo = getLang(tl);
              return (
                <div key={tl} style={{ borderTop: `1px solid ${t.border}`
                  <div style={{ display: "flex", alignItems: "center", ju
                    <span style={{ fontSize: 10, color: t.textFaint, font
                    {tr && <SpeakBtn text={tr} ttsLang={tlInfo.ttsCode} s
                  </div>
                  <textarea
                    style={{ width: "100%", background: "transparent", bo
                    value={tr || ""}
                    dir={tlInfo.dir}
                    rows={Math.max(2, Math.ceil((tr || "").length / 48))}
                    placeholder={s.typeOrTap}
                    onChange={e => setParagraphs(prev => prev.map(p => p.
                  />
                </div>
              );
            })}
          </div>
          {pi < paragraphs.length - 1 && <div style={{ height: 1, backgro
        </div>
      );
    })}
    <p style={{ fontSize: 12, color: t.textGhost, textAlign: "center" }}>
  </div>
)}
```

```
  {/* SAVED TAB */}
  {tab === "saved" && (
    <SavedTab
      saved={saved}
      t={t} s={s}
      search={savedSearch} setSearch={setSavedSearch}
      filter={savedFilter} setFilter={setSavedFilter}
      sort={savedSort} setSort={setSavedSort}
      editingWord={editingWord} setEditingWord={setEditingWord}
      onDelete={deleteSaved}
      onSetUserTr={setUserTranslation}
      onUpdate={updateSavedWord}
      onFetchExampleTr={fetchExampleTranslation}
      onOpenFlash={() => setFlashMode(true)}
      onOpenCopy={() => setShowCopy(true)}
      onClearAll={() => setSaved([])}
    />
  )}

  {/* QUIZ TAB */}
  {tab === "quiz" && (
    <Quiz
      saved={saved}
      t={t} s={s}
      prefs={quizPrefs} setPrefs={setQuizPrefs}
      onCorrect={onQuizCorrect}
      onWrong={onQuizWrong}
    />
  )}
</main>

{/* Bottom nav */}
<nav style={{ position: "fixed", bottom: 0, zIndex: 20, left: "50%", transf
  {([
    { id: "input" as const, Icon: FileText, label: s.text },
    { id: "reader" as const, Icon: BookOpen, label: s.reader },
    { id: "saved" as const, Icon: Star, label: `${s.saved} (${saved.length}
    { id: "quiz" as const, Icon: Brain, label: s.quiz },
  ]).map(({ id, Icon, label }) => (
    <button key={id} onClick={() => setTab(id)} style={{ flex: 1, padding:
      <Icon size={20} />
      <span style={{ fontSize: 10 }}>{label}</span>
    </button>
  ))}
</nav>

{/* Sheet overlay */}
```

```
{sheetOpen && <div onClick={() => setSheetOpen(false)} style={{ position: "

{/* Word detail sheet */}
<div onClick={e => e.stopPropagation()} style={{
  position: "fixed", bottom: 0, left: "50%",
  transform: sheetOpen ? "translateX(-50%) translateY(0)" : "translateX(-50
  width: "92%", maxWidth: 540, zIndex: 40,
  background: t.sheetBg, borderRadius: "20px 20px 0 0",
  padding: "12px 18px 28px", display: "flex", flexDirection: "column", gap:
  transition: "transform 0.3s ease-out", maxHeight: "85vh", overflowY: "aut
  boxShadow: isDark ? "0 -4px 40px #00000080" : "0 -4px 40px #00000020",
}}>
  {sheet && (() => {
    const srcInfo = getLang(sheet.srcLang);
    const wordTargets = Object.keys(sheet.translations).length
      ? Object.keys(sheet.translations)
      : targetLangs.filter(tl => tl !== sheet.srcLang);
    return (
      <>
        <div style={{ width: 40, height: 4, borderRadius: 2, background: t.

        {/* Word header */}
        <div style={{ background: t.card2, borderRadius: 14, border: `1px s
          <div style={{ display: "flex", alignItems: "center", justifyConte
            <div style={{ display: "flex", alignItems: "center", gap: 10, f
              <span style={{ fontSize: 28, fontWeight: 700, color: t.text }
              <SpeakBtn text={sheet.word} ttsLang={srcInfo.ttsCode} size={1
            </div>
            <span style={{ fontSize: 11, padding: "3px 10px", borderRadius:
              {srcInfo.flag} {srcInfo.name}
            </span>
          </div>
          <div style={{ height: 1, background: t.border }} />

          {/* Translations vertically; grid on wider screens */}
          {sheetLoading ? (
            <div style={{ padding: "16px", display: "flex", alignItems: "ce
              <span style={{ display: "inline-block", width: 16, height: 16
              {s.translating}
            </div>
          ) : (
            <div style={{
              padding: "10px 12px",
              display: "grid",
              gridTemplateColumns: wordTargets.length > 2 ? "repeat(auto-fi
              gap: 6,
            }}>
```

```
                {wordTargets.map(tl => {
                  const tlInfo = getLang(tl);
                  const tr = sheet.translations[tl];
                  return (
                    <div key={tl} style={{ padding: "8px 10px", display: "fle
                      <span style={{ fontSize: 11, color: t.textFaint, minWid
                      <span style={{ fontSize: 18, fontWeight: 700, color: "#
                        {tr || "—"}{tr && sheet.emoji ? ` ${sheet.emoji}` : "
                      </span>
                      {tr && <SpeakBtn text={tr} ttsLang={tlInfo.ttsCode} siz
                    </div>
                  );
                })}
              </div>
            )}
          </div>

          {/* Pronunciation, Synonym, Type pills */}
          <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fi
            {sheet.pronunciation && (
              <div style={{ background: t.pillBg, border: `1px solid ${t.bord
                <span style={{ fontSize: 10, color: t.textDim }}>{s.pronuncia
                <span style={{ fontSize: 14, fontWeight: 600, color: t.text,
              </div>
            )}
            {sheet.synonym && (
              <div style={{ background: t.pillBg, border: `1px solid ${t.bord
                <span style={{ fontSize: 10, color: t.textDim }}>{s.synonym}<
                <span style={{ fontSize: 14, fontWeight: 600, color: t.text,
              </div>
            )}
            {sheet.pos && (
              <div style={{ background: t.pillBg, border: `1px solid ${t.bord
                <span style={{ fontSize: 10, color: t.textDim }}>{s.type}</sp
                <span style={{ fontSize: 14, fontWeight: 600, color: posColor
              </div>
            )}
          </div>

          <button onClick={toggleSave} style={{ padding: 13, borderRadius: 12
            {isSaved ? `× ${s.remove}` : `✦ ${s.save}`}
          </button>
        </>
      );
    })()}
  </div>
</div>
```

```
  );
}

// ——— Add language picker (single-shot dropdown becomes a target after pick) ———

function AddLangButton({ onPick, options, t, label, chooseLabel }: {
  onPick: (code: string) => void;
  options: string[];
  t: Theme;
  label: string;
  chooseLabel: string;
}) {
  const [open, setOpen] = useState(false);
  const ref = useRef<HTMLDivElement | null>(null);
  useEffect(() => {
    function onDoc(e: MouseEvent) {
      if (ref.current && !ref.current.contains(e.target as Node)) setOpen(false);
    }
    document.addEventListener("mousedown", onDoc);
    return () => document.removeEventListener("mousedown", onDoc);
  }, []);
  if (options.length === 0) return null;
  return (
    <div ref={ref} style={{ position: "relative" }}>
      <button
        onClick={() => setOpen(o => !o)}
        style={{
          width: "100%", display: "flex", alignItems: "center", justifyContent: "
          gap: 6, padding: "10px 12px", borderRadius: 10,
          border: `1px dashed ${t.inputBorder}`, background: "transparent",
          color: t.textMuted, fontSize: 13, cursor: "pointer",
        }}
      >
        <Plus size={14} /> {label}
      </button>
      {open && (
        <div style={{
          position: "absolute", top: "calc(100% + 4px)", left: 0, right: 0, zInde
          background: t.card, border: `1px solid ${t.border}`, borderRadius: 10,
          maxHeight: 280, overflowY: "auto", boxShadow: "0 8px 30px rgba(0,0,0,.2
        }}>
          <div style={{ padding: "8px 12px", fontSize: 11, color: t.textDim, bord
          {options.map(code => {
            const lang = getLang(code);
            return (
              <button
                key={code}
```

```
                onClick={() => { onPick(code); setOpen(false); }}
                style={{
                  display: "block", width: "100%", textAlign: "left", padding: "1
                  background: "transparent", border: "none", color: t.text, fontS
                }}
              >
                <span style={{ marginRight: 8 }}>{lang.flag}</span>
                {lang.name} <span style={{ color: t.textDim, fontSize: 12 }}>· {l
              </button>
            );
          })}
        </div>
      )}
    </div>
  );
}

// ─── SavedTab ──────────────────────────────────────────────────────────────

type SavedTabProps = {
  saved: WordDetail[];
  t: Theme;
  s: Record<string, string>;
  search: string; setSearch: (v: string) => void;
  filter: "all" | "review" | "edited"; setFilter: (v: "all" | "review" | "edited"
  sort: "newest" | "alpha" | "wrong"; setSort: (v: "newest" | "alpha" | "wrong")
  editingWord: { word: string; srcLang: string } | null;
  setEditingWord: (w: { word: string; srcLang: string } | null) => void;
  onDelete: (word: string, srcLang: string) => void;
  onSetUserTr: (word: string, srcLang: string, tl: string, value: string) => void
  onUpdate: (word: string, srcLang: string, patch: Partial<WordDetail>) => void;
  onFetchExampleTr: (word: string, srcLang: string, tl: string) => Promise<void>;
  onOpenFlash: () => void;
  onOpenCopy: () => void;
  onClearAll: () => void;
};

function SavedTab({
  saved, t, s, search, setSearch, filter, setFilter, sort, setSort,
  editingWord, setEditingWord,
  onDelete, onSetUserTr, onUpdate, onFetchExampleTr,
  onOpenFlash, onOpenCopy, onClearAll,
}: SavedTabProps) {
  const filtered = saved.filter(w => {
    if (filter === "review" && !w.needsReview) return false;
    if (filter === "edited" && !w.userTranslations) return false;
    if (search.trim()) {
```

```
    const q = search.trim().toLowerCase();
    const haystacks = [w.word, ...Object.values(w.translations), ...(w.userNote
    if (!haystacks.some(h => h && h.toLowerCase().includes(q))) return false;
  }
  return true;
});
const sorted = filtered.slice().sort((a, b) => {
  if (sort === "alpha") return a.word.localeCompare(b.word);
  if (sort === "wrong") return (b.errorCount || 0) - (a.errorCount || 0);
  return (b.savedAt || 0) - (a.savedAt || 0);
});

const reviewCount = saved.filter(w => w.needsReview).length;
const editedCount = saved.filter(w => w.userTranslations).length;

return (
  <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
    <div style={{ display: "flex", justifyContent: "space-between", alignItems:
      <span style={{ fontSize: 13, color: t.textDim }}>{saved.length} {s.countS
      {saved.length > 0 && (
        <div style={{ display: "flex", gap: 8, flexWrap: "wrap" }}>
          <button onClick={onOpenFlash} style={{ fontSize: 11, padding: "4px 10
          <button onClick={onOpenCopy} style={{ fontSize: 11, padding: "4px 10p
          <button onClick={onClearAll} style={{ fontSize: 11, padding: "4px 10p
        </div>
      )}
    </div>

    {saved.length > 0 && (
      <>
        <div style={{ position: "relative" }}>
          <Search size={14} style={{ position: "absolute", left: 12, top: "50%"
          <input
            value={search}
            onChange={e => setSearch(e.target.value)}
            placeholder={s.searchSaved}
            style={{ width: "100%", padding: "10px 12px 10px 34px", borderRadiu
          />
        </div>

        <div style={{ display: "flex", gap: 6, flexWrap: "wrap" }}>
          {([
            { id: "all" as const, label: `${s.filterAll} (${saved.length})` },
            { id: "review" as const, label: `${s.filterReview} (${reviewCount})
            { id: "edited" as const, label: `${s.filterEdited} (${editedCount})
          ]).map(c => (
            <button key={c.id} onClick={() => setFilter(c.id)}
```

```
                style={{
                  fontSize: 11, padding: "4px 10px", borderRadius: 20,
                  border: `1px solid ${filter === c.id ? "#60a5fa" : t.border}`,
                  background: filter === c.id ? "#60a5fa22" : "transparent",
                  color: filter === c.id ? "#60a5fa" : t.textDim,
                  cursor: "pointer",
                }}>{c.label}</button>
            ))}
            <select value={sort} onChange={e => setSort(e.target.value as "newest
              style={{ marginLeft: "auto", fontSize: 11, padding: "4px 8px", bord
              <option value="newest">{s.sortNewest}</option>
              <option value="alpha">{s.sortAlpha}</option>
              <option value="wrong">{s.sortMostWrong}</option>
            </select>
          </div>
        </>
      )}

      {saved.length === 0
        ? <p style={{ fontSize: 14, color: t.textDim, textAlign: "center", lineHe
        : sorted.length === 0
          ? <p style={{ fontSize: 13, color: t.textFaint, textAlign: "center", ma
          : sorted.map(w => (
            <SavedCard key={`${w.word}-${w.srcLang}`}
              w={w} t={t} s={s}
              isEditing={!!(editingWord && editingWord.word === w.word && editing
              setEditing={(on) => setEditingWord(on ? { word: w.word, srcLang: w.
              onDelete={() => onDelete(w.word, w.srcLang)}
              onSetUserTr={(tl, v) => onSetUserTr(w.word, w.srcLang, tl, v)}
              onUpdate={(p) => onUpdate(w.word, w.srcLang, p)}
              onFetchExampleTr={(tl) => onFetchExampleTr(w.word, w.srcLang, tl)}
            />
          ))
      }
    </div>
  );
}

function SavedCard({ w, t, s, isEditing, setEditing, onDelete, onSetUserTr, onUpd
  w: WordDetail;
  t: Theme;
  s: Record<string, string>;
  isEditing: boolean;
  setEditing: (on: boolean) => void;
  onDelete: () => void;
  onSetUserTr: (tl: string, v: string) => void;
  onUpdate: (p: Partial<WordDetail>) => void;
```

```
  onFetchExampleTr: (tl: string) => Promise<void>;
}) {
  const srcInfo = getLang(w.srcLang);
  const wordTargets = Object.keys(w.translations);
  const [showNote, setShowNote] = useState(false);
  const [noteDraft, setNoteDraft] = useState(w.userNote || "");
  const [showExample, setShowExample] = useState(false);
  useEffect(() => { setNoteDraft(w.userNote || ""); }, [w.userNote]);

  return (
    <div style={{ background: t.card, borderRadius: 12, padding: "12px 14px", bor
      <div style={{ display: "flex", alignItems: "center", justifyContent: "space
        <div style={{ display: "flex", alignItems: "center", gap: 8, flexWrap: "w
          <span style={{ fontSize: 20, fontWeight: 700, color: t.text, direction:
          <SpeakBtn text={w.word} ttsLang={srcInfo.ttsCode} size={12} color={t.te
          <span style={{ fontSize: 10, padding: "2px 8px", borderRadius: 20, back
            {srcInfo.flag} {srcInfo.name}{w.pos ? ` · ${w.pos}` : ""}
          </span>
          {w.needsReview && (
            <span style={{ fontSize: 10, padding: "2px 8px", borderRadius: 20, ba
          )}
          {w.userTranslations && (
            <span style={{ fontSize: 10, padding: "2px 8px", borderRadius: 20, ba
          )}
        </div>
        <div style={{ display: "flex", gap: 4 }}>
          <button onClick={() => setEditing(!isEditing)} title={s.editTranslation
            style={{ padding: 6, borderRadius: 8, border: `1px solid ${isEditing
            <Edit3 size={13} />
          </button>
          <button onClick={() => setShowNote(v => !v)} title={s.addNote}
            style={{ padding: 6, borderRadius: 8, border: `1px solid ${w.userNote
            <MessageSquare size={13} />
          </button>
          <button onClick={onDelete} style={{ padding: 6, borderRadius: 8, border
        </div>
      </div>

      {(w.synonym || w.pronunciation) && (
        <div style={{ display: "flex", flexWrap: "wrap", gap: 12, fontSize: 12, c
          {w.synonym && <span>≈ <span style={{ direction: srcInfo.dir, fontStyle:
          {w.pronunciation && <span style={{ fontFamily: "monospace" }}>/{w.pronu
        </div>
      )}

      <div style={{
        display: "grid",
```

```
  gridTemplateColumns: wordTargets.length > 2 ? "repeat(auto-fit, minmax(22
  gap: 6, marginTop: 2,
}}>
  {wordTargets.map(tl => {
    const original = w.translations[tl];
    const userTr = w.userTranslations?.[tl];
    const tr = userTr ?? original;
    const tlInfo = getLang(tl);
    if (!tr) return null;
    return (
      <div key={tl} style={{ display: "flex", alignItems: "flex-start", gap
        <span style={{ fontSize: 10, color: t.textFaint, minWidth: 52, padd
        {isEditing ? (
          <EditableTranslation
            value={userTr ?? original}
            original={original}
            edited={!!userTr}
            dir={tlInfo.dir}
            t={t} s={s}
            onSave={(v) => onSetUserTr(tl, v)}
            onReset={() => onSetUserTr(tl, "")}
          />
        ) : (
          <>
            <span style={{ fontSize: 16, color: userTr ? "#22c55e" : "#60a5
              {tr}{w.emoji ? ` ${w.emoji}` : ""}
            </span>
            <SpeakBtn text={tr} ttsLang={tlInfo.ttsCode} size={11} />
          </>
        )}
      </div>
    );
  })}
</div>

{/* Note */}
{(showNote || (w.userNote && !isEditing)) && (
  <div style={{ background: t.card2, borderRadius: 8, padding: "8px 10px",
    {showNote ? (
      <>
        <textarea
          value={noteDraft}
          onChange={e => setNoteDraft(e.target.value)}
          placeholder={s.yourNote}
          rows={2}
          style={{ width: "100%", background: "transparent", border: "none"
        />
```

```
        <div style={{ display: "flex", justifyContent: "flex-end", gap: 6,
          <button onClick={() => { setNoteDraft(w.userNote || ""); setShowN
            style={{ fontSize: 11, padding: "3px 10px", borderRadius: 8, bo
          <button onClick={() => { onUpdate({ userNote: noteDraft.trim() ||
            style={{ fontSize: 11, padding: "3px 10px", borderRadius: 8, bo
        </div>
      </>
    ) : (
      <span style={{ fontSize: 13, color: t.textMuted, fontStyle: "italic",
    )}
  </div>
)}

{/* Example sentence */}
{w.exampleSentence && (
  <div style={{ background: t.card2, borderRadius: 8, padding: "8px 10px",
    <div style={{ display: "flex", justifyContent: "space-between", alignIt
      <span style={{ fontSize: 11, color: t.textFaint, fontWeight: 600 }}>
      <div style={{ display: "flex", gap: 4 }}>
        <SpeakBtn text={w.exampleSentence} ttsLang={srcInfo.ttsCode} size={
        <button onClick={() => setShowExample(v => !v)} style={{ fontSize:
          {showExample ? "−" : "+"}
        </button>
      </div>
    </div>
    <span style={{ fontSize: 14, color: t.text, direction: srcInfo.dir, lin
    {showExample && (
      <div style={{ display: "flex", flexDirection: "column", gap: 4, margi
        {wordTargets.map(tl => {
          const tlInfo = getLang(tl);
          const tr = w.exampleTranslations?.[tl];
          return (
            <div key={tl} style={{ display: "flex", alignItems: "center", g
              <span style={{ fontSize: 10, color: t.textFaint, minWidth: 52
              {tr ? (
                <>
                  <span style={{ fontSize: 13, color: t.textDim, direction:
                  <SpeakBtn text={tr} ttsLang={tlInfo.ttsCode} size={10} />
                </>
              ) : (
                <button onClick={() => onFetchExampleTr(tl)} style={{ fontS
                  ✦ {s.translateSentence}
                </button>
              )}
            </div>
          );
        })}
```

```
            </div>
          )}
        </div>
      )}
    </div>
  );
}

function EditableTranslation({ value, original, edited, dir, t, s, onSave, onRese
  value: string; original: string; edited: boolean; dir: "ltr" | "rtl";
  t: Theme; s: Record<string, string>;
  onSave: (v: string) => void; onReset: () => void;
}) {
  const [draft, setDraft] = useState(value);
  useEffect(() => { setDraft(value); }, [value]);
  return (
    <div style={{ flex: 1, display: "flex", flexDirection: "column", gap: 4 }}>
      <input
        value={draft}
        onChange={e => setDraft(e.target.value)}
        dir={dir}
        style={{ width: "100%", padding: "6px 10px", borderRadius: 8, border: `1p
      />
      <div style={{ display: "flex", gap: 6, justifyContent: "flex-end" }}>
        {edited && (
          <button onClick={onReset} style={{ fontSize: 10, padding: "2px 8px", bo
        )}
        <button onClick={() => onSave(draft)} disabled={draft.trim() === value.tr
          style={{ fontSize: 10, padding: "2px 10px", borderRadius: 8, border: "1
          {s.saveEdit}
        </button>
      </div>
      {edited && <span style={{ fontSize: 10, color: t.textFaint }}>≈ {original}<
    </div>
  );
}

// ─── Confetti ──────────────────────────────────────────────────────────

function ConfettiBurst({ count = 40, full = false }: { count?: number; full?: boo
  const colors = ["#f97316", "#22c55e", "#60a5fa", "#a78bfa", "#fbbf24", "#ec4899
  const pieces = useMemo(() => Array.from({ length: count }, (_, i) => {
    const left = Math.random() * 100;
    const delay = Math.random() * (full ? 1.6 : 0.4);
    const dur = 1.8 + Math.random() * 1.4;
    const size = 6 + Math.random() * 10;
    const color = colors[i % colors.length];
```

```
    const rot = Math.random() * 360;
    const shape = Math.random() > 0.5 ? "50%" : "2px";
    return { left, delay, dur, size, color, rot, shape };
  }), [count, full]);
  return (
    <div style={{ position: full ? "fixed" : "absolute", inset: 0, pointerEvents:
      {pieces.map((p, i) => (
        <span key={i} style={{
          position: "absolute", top: 0, left: `${p.left}%`,
          width: p.size, height: p.size, background: p.color, borderRadius: p.sha
          transform: `rotate(${p.rot}deg)`,
          animation: `confetti-fall ${p.dur}s cubic-bezier(.4,.1,.5,1) ${p.delay}
        }} />
      ))}
    </div>
  );
}

// ─── Quiz ──────────────────────────────────────────────────────────────

type QuizQ = { w: WordDetail; tl: string; correct: string; choices?: string[] };

function Quiz({ saved, t, s, prefs, setPrefs, onCorrect, onWrong }: {
  saved: WordDetail[];
  t: Theme;
  s: Record<string, string>;
  prefs: QuizPrefs;
  setPrefs: (p: QuizPrefs) => void;
  onCorrect: (word: string, srcLang: string) => void;
  onWrong: (word: string, srcLang: string) => void;
}) {
  type Stage = "setup" | "playing" | "done";
  const [stage, setStage] = useState<Stage>("setup");
  const [count, setCount] = useState<number | "all">(10);
  const [questions, setQuestions] = useState<QuizQ[]>([]);
  const [qIdx, setQIdx] = useState(0);
  const [answer, setAnswer] = useState("");
  const [feedback, setFeedback] = useState<null | "correct" | "wrong">(null);
  const [hintShown, setHintShown] = useState(false);
  const [shake, setShake] = useState(false);
  const [pop, setPop] = useState(false);
  const [correctCount, setCorrectCount] = useState(0);
  const [wrongList, setWrongList] = useState<{ w: WordDetail; tl: string; correct

  // Build a deck of questions when starting
  function buildDeck(): QuizQ[] {
    const pool: QuizQ[] = [];
```

```
  // Spaced repetition: weight needsReview * 3, errorCount * 2, recent saves *
  const weighted: { w: WordDetail; weight: number }[] = [];
  for (const w of saved) {
    const langs = Object.keys(w.translations).filter(tl => tl !== w.srcLang);
    if (langs.length === 0) continue;
    const weight = (w.needsReview ? 3 : 1) + (w.errorCount || 0) * 2;
    weighted.push({ w, weight });
  }
  if (weighted.length === 0) return [];
  // Sample with weights without replacement
  const desired = count === "all" ? weighted.length : Math.min(count, weighted.
  const remaining = weighted.slice();
  while (pool.length < desired && remaining.length) {
    const total = remaining.reduce((a, b) => a + b.weight, 0);
    let r = Math.random() * total;
    let idx = 0;
    for (let i = 0; i < remaining.length; i++) {
      r -= remaining[i].weight;
      if (r <= 0) { idx = i; break; }
    }
    const picked = remaining.splice(idx, 1)[0].w;
    const langs = Object.keys(picked.translations).filter(tl => tl !== picked.s
    const tl = langs[Math.floor(Math.random() * langs.length)];
    const correct = effectiveTr(picked, tl);
    if (!correct) continue;
    let q: QuizQ = { w: picked, tl, correct };
    if (prefs.mode === "choice") {
      const distractors: string[] = [];
      const others = saved.filter(x => !(x.word === picked.word && x.srcLang ==
      for (let i = 0; i < 5 && distractors.length < 3; i++) {
        const cand = others[Math.floor(Math.random() * others.length)];
        if (!cand) break;
        const d = effectiveTr(cand, tl);
        if (d && d !== correct && !distractors.includes(d)) distractors.push(d)
      }
      const choices = [correct, ...distractors];
      for (let i = choices.length - 1; i > 0; i--) {
        const j = Math.floor(Math.random() * (i + 1));
        [choices[i], choices[j]] = [choices[j], choices[i]];
      }
      q.choices = choices;
    }
    pool.push(q);
  }
  return pool;
}
```

```
function start() {
  const deck = buildDeck();
  if (deck.length === 0) return;
  setQuestions(deck);
  setQIdx(0);
  setCorrectCount(0);
  setWrongList([]);
  setAnswer("");
  setFeedback(null);
  setHintShown(false);
  setStage("playing");
}

const cur = questions[qIdx];

function submitAnswer(given: string, forceCorrect = false) {
  if (!cur || feedback) return;
  const ok = forceCorrect || answersMatch(given, cur.correct, cur.tl, prefs.ign
  if (ok) {
    setFeedback("correct");
    setPop(true);
    setTimeout(() => setPop(false), 400);
    setCorrectCount(c => c + 1);
    onCorrect(cur.w.word, cur.w.srcLang);
    if (prefs.sound) playCorrectSound();
  } else {
    setFeedback("wrong");
    setShake(true);
    setTimeout(() => setShake(false), 500);
    setWrongList(arr => [...arr, { w: cur.w, tl: cur.tl, correct: cur.correct,
    onWrong(cur.w.word, cur.w.srcLang);
    if (prefs.sound) playWrongSound();
  }
}

function nextQuestion() {
  if (qIdx + 1 >= questions.length) {
    setStage("done");
    if (prefs.sound) playFinishSound();
  } else {
    setQIdx(i => i + 1);
    setAnswer("");
    setFeedback(null);
    setHintShown(false);
  }
}
```

```
function markCorrect() {
  if (!cur) return;
  // Override Google's translation with the user's answer
  if (answer.trim() && answer.trim() !== cur.correct) {
    const userTr = { ...(cur.w.userTranslations || {}), [cur.tl]: answer.trim()
    cur.w.userTranslations = userTr;
  }
  setFeedback(null);
  setWrongList(arr => arr.filter(x => !(x.w.word === cur.w.word && x.tl === cur
  submitAnswer(answer, true);
}

function skip() {
  if (!cur) return;
  setWrongList(arr => [...arr, { w: cur.w, tl: cur.tl, correct: cur.correct, gi
  onWrong(cur.w.word, cur.w.srcLang);
  nextQuestion();
}

// ─── Render: SETUP ────────
if (stage === "setup") {
  const eligible = saved.filter(w => Object.keys(w.translations).some(tl => tl
  if (eligible.length === 0) {
    return (
      <div style={{ display: "flex", flexDirection: "column", gap: 14, alignIte
        <Brain size={48} color={t.textFaint} />
        <p style={{ color: t.textDim, fontSize: 14 }}>{s.nothingSaved}</p>
      </div>
    );
  }
  const opts: (number | "all")[] = [5, 10, 15, 20, "all"];
  return (
    <div style={{ display: "flex", flexDirection: "column", gap: 18 }}>
      <div style={{ display: "flex", flexDirection: "column", gap: 6, alignItem
        <Brain size={36} color="#60a5fa" />
        <h2 style={{ fontSize: 20, color: t.text, margin: 0 }}>{s.quizSetupTitl
        <p style={{ fontSize: 13, color: t.textDim, margin: 0 }}>{eligible.leng
      </div>

      <div style={{ display: "flex", flexWrap: "wrap", gap: 8, justifyContent:
        {opts.map(o => (
          <button key={String(o)} onClick={() => setCount(o)}
            style={{
              padding: "10px 18px", borderRadius: 12, fontSize: 16, fontWeight:
              border: `1px solid ${count === o ? "#60a5fa" : t.border}`,
              background: count === o ? "#60a5fa22" : t.card2,
              color: count === o ? "#60a5fa" : t.textMuted,
```

```
              minWidth: 60,
            }}>
            {o === "all" ? s.quizAll : o}
          </button>
        ))}
      </div>

      <div style={{ display: "flex", flexDirection: "column", gap: 8 }}>
        <span style={{ fontSize: 12, color: t.textDim, fontWeight: 600 }}>{s.qu
        <div style={{ display: "flex", gap: 8, flexWrap: "wrap" }}>
          {([
            { id: "type" as const, label: s.modeTyping },
            { id: "choice" as const, label: s.modeChoice },
            { id: "reverse" as const, label: s.modeReverse },
          ]).map(m => (
            <button key={m.id} onClick={() => setPrefs({ ...prefs, mode: m.id }
              style={{
                flex: "1 1 auto", minWidth: 100, padding: "10px 12px", borderRa
                border: `1px solid ${prefs.mode === m.id ? "#60a5fa" : t.border
                background: prefs.mode === m.id ? "#60a5fa22" : t.card2,
                color: prefs.mode === m.id ? "#60a5fa" : t.textMuted,
              }}>{m.label}</button>
          ))}
        </div>
      </div>

      <div style={{ display: "flex", flexDirection: "column", gap: 8 }}>
        <label style={{ fontSize: 12, color: t.textDim, display: "flex", gap: 8
          <input type="checkbox" checked={prefs.ignoreDiacritics} onChange={e =
          {s.quizIgnoreDiacritics}
        </label>
        <label style={{ fontSize: 12, color: t.textDim, display: "flex", gap: 8
          <input type="checkbox" checked={prefs.sound} onChange={e => setPrefs(
          🔊 {prefs.sound ? s.soundOn : s.soundOff}
        </label>
        <label style={{ fontSize: 12, color: t.textDim, display: "flex", gap: 8
          <input type="checkbox" checked={prefs.timer} onChange={e => setPrefs(
          ⏱️ {s.timer}
        </label>
      </div>

      <button onClick={start} style={{ padding: 14, borderRadius: 12, border: "
        {s.quizStart} →
      </button>
    </div>
  );
}
```

```
// ─── Render: DONE ────────
if (stage === "done") {
  const total = questions.length;
  const acc = total ? Math.round((correctCount / total) * 100) : 0;
  const great = acc >= 80;
  const good = acc >= 50;
  return (
    <div style={{ display: "flex", flexDirection: "column", gap: 18, alignItems
      <ConfettiBurst count={70} full />
      <span style={{ fontSize: 70, animation: "trophy-bounce .9s ease-out" }}>🏆
      <h2 style={{ fontSize: 28, color: t.text, margin: 0 }}>{s.quizFinishTitle
      <p style={{ fontSize: 16, color: great ? "#22c55e" : good ? "#60a5fa" : "
        {great ? s.quizCelebrateGreat : good ? s.quizCelebrateGood : s.quizCele
      </p>
      <div style={{ background: t.card, borderRadius: 14, padding: "18px 24px",
        <div style={{ display: "flex", flexDirection: "column", alignItems: "ce
          <span style={{ fontSize: 28, fontWeight: 700, color: "#22c55e" }}>{co
          <span style={{ fontSize: 11, color: t.textFaint }}>{s.quizCorrectCoun
        </div>
        <div style={{ display: "flex", flexDirection: "column", alignItems: "ce
          <span style={{ fontSize: 28, fontWeight: 700, color: "#ef4444" }}>{to
          <span style={{ fontSize: 11, color: t.textFaint }}>{s.quizWrongCount}
        </div>
        <div style={{ display: "flex", flexDirection: "column", alignItems: "ce
          <span style={{ fontSize: 28, fontWeight: 700, color: "#60a5fa" }}>{ac
          <span style={{ fontSize: 11, color: t.textFaint }}>{s.quizAccuracy}</
        </div>
      </div>

      {wrongList.length > 0 && (
        <div style={{ width: "100%", display: "flex", flexDirection: "column",
          <span style={{ fontSize: 12, color: t.textDim, fontWeight: 600 }}>{s.
          {wrongList.map((x, i) => {
            const tlInfo = getLang(x.tl);
            const srcInfo = getLang(x.w.srcLang);
            return (
              <div key={i} style={{ background: t.card, borderRadius: 10, paddi
                <span style={{ fontSize: 14, color: t.text, direction: srcInfo.
                <span style={{ fontSize: 12, color: t.textDim }}>
                  {tlInfo.flag} <span style={{ color: "#22c55e" }}>{x.correct}<
                  {x.given && x.given !== "—" && <span style={{ color: t.textFa
                </span>
              </div>
            );
          })}
        </div>
```

```
      )}

      <div style={{ display: "flex", gap: 10, marginTop: 6 }}>
        <button onClick={() => setStage("setup")} style={{ padding: "12px 20px"
          <RefreshCw size={14} style={{ verticalAlign: "middle", marginRight: 4
        </button>
        <button onClick={() => setStage("setup")} style={{ padding: "12px 20px"
          <Award size={14} style={{ verticalAlign: "middle", marginRight: 4 }}
        </button>
      </div>
    </div>
  );
}

// ─── Render: PLAYING ────────
if (!cur) return null;
const srcInfo = getLang(cur.w.srcLang);
const tlInfo = getLang(cur.tl);
const isReverse = prefs.mode === "reverse";
const promptText = isReverse ? cur.correct : cur.w.word;
const promptLang = isReverse ? tlInfo : srcInfo;
const answerLang = isReverse ? srcInfo : tlInfo;
const expected = isReverse ? cur.w.word : cur.correct;

function checkReverse(given: string) {
  if (feedback) return;
  const ok = answersMatch(given, expected, isReverse ? cur.w.srcLang : cur.tl,
  if (ok) {
    setFeedback("correct");
    setPop(true); setTimeout(() => setPop(false), 400);
    setCorrectCount(c => c + 1);
    onCorrect(cur.w.word, cur.w.srcLang);
    if (prefs.sound) playCorrectSound();
  } else {
    setFeedback("wrong");
    setShake(true); setTimeout(() => setShake(false), 500);
    setWrongList(arr => [...arr, { w: cur.w, tl: cur.tl, correct: expected, giv
    onWrong(cur.w.word, cur.w.srcLang);
    if (prefs.sound) playWrongSound();
  }
}

return (
  <div style={{ display: "flex", flexDirection: "column", gap: 14, position: "r
    {/* Progress bar */}
    <div style={{ display: "flex", alignItems: "center", gap: 10 }}>
      <div style={{ flex: 1, height: 8, background: t.card2, borderRadius: 4, o
```

```
    <div style={{ width: `${((qIdx) / questions.length) * 100}%`, height: "
  </div>
  <span style={{ fontSize: 12, color: t.textDim, minWidth: 50, textAlign: "
</div>

{/* Score chips */}
<div style={{ display: "flex", gap: 8, justifyContent: "center" }}>
  <span style={{ fontSize: 11, padding: "3px 10px", borderRadius: 20, backg
  <span style={{ fontSize: 11, padding: "3px 10px", borderRadius: 20, backg
</div>

{/* Question card */}
<div className={`${shake ? "shake" : ""} ${feedback === "correct" ? "flash-
  style={{ background: t.card, borderRadius: 16, border: `1px solid ${t.bor
  {feedback === "correct" && <ConfettiBurst count={20} />}
  <span style={{ fontSize: 11, padding: "3px 10px", borderRadius: 20, backg
    {promptLang.flag} {promptLang.name} → {answerLang.flag} {answerLang.nam
  </span>
  <div className={pop ? "pop" : ""} style={{ display: "flex", alignItems: "
    <span style={{ fontSize: 32, fontWeight: 700, color: t.text, direction:
    <SpeakBtn text={promptText} ttsLang={promptLang.ttsCode} size={18} />
  </div>
  {hintShown && cur.w.exampleSentence && (
    <span style={{ fontSize: 13, color: t.textDim, fontStyle: "italic", dir
  )}
  {hintShown && !cur.w.exampleSentence && cur.w.synonym && (
    <span style={{ fontSize: 13, color: t.textDim, fontStyle: "italic" }}>
  )}
  {hintShown && !cur.w.exampleSentence && !cur.w.synonym && (
    <span style={{ fontSize: 13, color: t.textDim, fontStyle: "italic" }}>
  )}

  {/* Answer area */}
  {prefs.mode === "choice" ? (
    <div style={{ display: "flex", flexDirection: "column", gap: 8, width:
      {(cur.choices || []).map((c, i) => {
        const isCorrect = answersMatch(c, cur.correct, cur.tl, prefs.ignore
        const picked = answer === c;
        const showState = feedback && (picked || isCorrect);
        const bg = !showState ? t.card2
          : isCorrect ? "#22c55e22"
          : picked ? "#ef444422" : t.card2;
        const color = !showState ? t.text
          : isCorrect ? "#22c55e"
          : picked ? "#ef4444" : t.text;
        const border = !showState ? t.border
          : isCorrect ? "#22c55e"
```

```
            : picked ? "#ef4444" : t.border;
          return (
            <button key={i} disabled={!!feedback} onClick={() => { setAnswer(
              style={{ padding: "12px 14px", borderRadius: 10, border: `1px s
              {c}
            </button>
          );
        })}
      </div>
    ) : (
      <input
        value={answer}
        onChange={e => setAnswer(e.target.value)}
        onKeyDown={e => { if (e.key === "Enter") { isReverse ? checkReverse(a
        placeholder={s.quizYourAnswer}
        dir={answerLang.dir}
        disabled={!!feedback}
        style={{ width: "100%", padding: "12px 14px", borderRadius: 10, borde
      />
    )}

    {feedback === "correct" && (
      <div style={{ fontSize: 36, animation: "pop .4s ease-out" }}>✅</div>
    )}
    {feedback === "wrong" && (
      <div style={{ display: "flex", flexDirection: "column", alignItems: "ce
        <span style={{ fontSize: 36 }}>😢</span>
        <span style={{ fontSize: 13, color: t.textDim }}>{s.quizCorrect}: <st
      </div>
    )}
  </div>

  {/* Actions */}
  {!feedback && prefs.mode !== "choice" && (
    <div style={{ display: "flex", gap: 8 }}>
      <button onClick={() => setHintShown(true)} disabled={hintShown}
        style={{ flex: 1, padding: 12, borderRadius: 10, border: `1px solid $
      <button onClick={skip}
        style={{ flex: 1, padding: 12, borderRadius: 10, border: `1px solid $
      <button onClick={() => isReverse ? checkReverse(answer) : submitAnswer(
        style={{ flex: 2, padding: 12, borderRadius: 10, border: "none", back
        ✓ {s.quizCheck}
      </button>
    </div>
  )}

  {feedback && (
```

```
        <div style={{ display: "flex", gap: 8 }}>
          {feedback === "wrong" && (
            <button onClick={markCorrect}
              style={{ flex: 1, padding: 12, borderRadius: 10, border: "1px solid
              ✓ {s.quizMarkRight}
            </button>
          )}
          <button onClick={nextQuestion}
            style={{ flex: 2, padding: 12, borderRadius: 10, border: "none", back
            {s.quizNext}
          </button>
        </div>
      )}
    </div>
  );
}
```